

Symmetry and AI-assisted discovery of magic-state factories

Shubham P. Jain,^{1,2,*} Adam Wills,^{2,3,†} and Shraddha Singh^{2,‡}

¹*Joint Center for Quantum Information and Computer Science,
NIST/University of Maryland, College Park, Maryland 20742, USA*

²*IBM Quantum, IBM T.J. Watson Research Center, Yorktown Heights, New York 10598, USA*

³*Center for Theoretical Physics, a Leinweber Institute,
Massachusetts Institute of Technology, Cambridge, Massachusetts 02139, USA*

(Dated: October 6, 2026)

Magic-state distillation is a major resource cost in fault-tolerant quantum computing. The cost of a magic-state factory depends strongly on its failure rate, which grows with the number of input magic states. Although symmetry-restricted methods have recently made distance two searches tractable, distance three and above have remained elusive at moderate input counts. We develop symmetry- and AI-assisted methods to search this regime. We present a unified binary-matrix formulation encompassing both triorthogonal-code and direct circuit searches. We show that distance at least three is equivalent to nonzero, pairwise distinct syndromes, separating the choice of syndromes from the search for compatible output gates. We restrict the syndrome search using group symmetry and language-model agents, followed by deterministic solving and independent verification. Our searches yield 699 factory classes, including 564 new ones. These include factories for pure-T states and factories with entangled outputs comprising combinations of T, CS, and CCZ magic states. The pure-T factories $[[63, 11, 3]]$ and $[[850, 128, 6]]$ achieve the lowest overhead exponents we know among protocols with at most 100 and 1000 inputs, respectively, with $\gamma = 1.589$ and $\gamma = 1.057$. Our $[[1715, 287, 6]]$ factory, with $\gamma = 0.998$, is the smallest known pure-T factory with $\gamma < 1$. We also provide a context directory of search briefs and campaign notes with which readers can train their own agents and tailor the search to their requirements. With these results, we begin constructing an active, open-source repository of magic-state distillation protocols for the quantum community, supplemented by our methods and data, for the practical fault-tolerant quantum computing regime.

I. INTRODUCTION

Fault-tolerant quantum computation protects logical information using quantum error-correcting codes [1–3]. On most practically accessible codes [4], the Clifford gates, generated by the Hadamard, the phase gate $S = \text{diag}(1, i)$ and the CNOT, can often be implemented by transversal gates or lattice surgery [5, 6]. But no code that detects an arbitrary single-qubit error has a universal transversal gate set [7]. A common way to achieve universality fault-tolerantly is to supply a non-Clifford gate U through gate teleportation [8, 9]. Gate teleportation is a Clifford circuit by which the *magic state* $|U\rangle$, obtained by applying U to $|+\rangle$ states, is *injected* to apply U . The teleported gate is fault-tolerant when the Clifford circuit is fault-tolerant, which is generally trivial, and the magic state is of high-quality. Supplying such states is the problem that *magic-state distillation* [10, 11] was designed to solve. It converts many low-quality magic states into fewer high-quality ones. Resource estimates attribute a large share of a machine’s qubits and time to distillation [12, 13].

The magic states most studied in the distillation literature are $|T\rangle$, $|CS\rangle$ and $|CCZ\rangle$. They teleport the $T = \text{diag}(1, e^{i\pi/4})$ gate, the $CS = \text{diag}(1, 1, 1, i)$ gate and

the doubly controlled Z gate CCZ , respectively. Among these, a noisy T state is the easiest to encode in most practical codes [14–17]. We therefore concentrate on factories whose inputs are T states. For efficiency of the compiled circuits, however, we would like to analyze as large a set of outputs as possible. We restrict to the third level of the *Clifford hierarchy* [8, 18]. These are the gates that map Pauli operators to Clifford operators under conjugation, and their gate teleportation circuit is trivial.¹ Importantly, our set of targets spans every magic state required to teleport a diagonal gate in the third level of the Clifford hierarchy [18].

The original idea behind a distillation circuit is inspired by QEC codes with transversal implementation of non-Clifford gates. It consumes n noisy input magic states, each with an independent and identically distributed error of probability p , on which r *parity checks* are measured, each the parity of the faults on some subset of the inputs. The *distance* d is the smallest number of faulty input magic states that pass every check and still corrupt the output, thus yielding an error scaling of $O(p^d)$. If every single fault is detected, the rejection probability is approximately np when $np \ll 1$. We write such a factory as $[[n, k, d]]$, and its *width* $N = k + r$ counts the logical qubits, or *wires*, it acts on. For pure-T outputs the literature uses a standard figure of merit, the

* shubhamj810@gmail.com

† a_wills@mit.edu

‡ shraddha.s@ibm.com

¹ States in deeper levels of the Clifford hierarchy require, with some probability, non-Clifford corrections for gate teleportation to succeed.

overhead exponent γ [19, 20]. Reaching an output error ϵ by concatenating a factory with itself costs

$$O(\log^\gamma(1/\epsilon)), \quad \gamma = \frac{\log(n/k)}{\log d}, \quad (1)$$

inputs per output. For a fixed output target and required distance, we therefore minimize the input count n .

The first factory distilled fifteen noisy $|T\rangle$ states into one on the 15-qubit Reed–Muller code, whose T is transversal [11]. Bravyi and Haah reduced the code requirement to a combinatorial condition. A factory is a binary *triorthogonal* matrix with prescribed single, pair and triple overlap parities of its rows [19]. Haah and Hastings extended the condition to CS and CCZ outputs and built families by puncturing Reed–Muller codes [20]. Richer protocols include Toffoli-state distillation [21, 22], *synthillation*, which directly distills the gate a circuit needs rather than the T states it would be built from [23, 24], *catalysis*, in which a magic state assists a conversion and is returned intact [25, 26], and codes over binary extension fields [27]. Existing approaches include asymptotic constructions, exhaustive classifications at small sizes, and searches within specified code families. Asymptotic constructions reach constant space overhead at large block lengths [28–31], and architectural studies estimate the cost of surface-code layouts and injections [14–17, 32–39]. At small sizes, exhaustive classification is possible. Nezami and Haah enumerated all triorthogonal codes with $n + k \leq 38$, which are the pure- T factories [40]. Our companion paper extends the enumeration to $n \leq 54$ and to every third-level output [41]. Other searches use particular code structures. Doubling and code conversion make transversal- T codes from transversal-Clifford codes [42–44], borrowed identities carve a factory from a circuit that acts as the identity on $|+\rangle^{\otimes N}$ [45], and satisfiability and integer-programming solvers map compact factories at small widths [46, 47].

We search beyond the classified range for factories with tens to hundreds of inputs, delivering T states and entangled states at the third level of the Clifford hierarchy [8, 9], at distance three and above. Efficient searches in the borrowed-identity framework already chart the distance-two landscape [45, 48], and cultivation [34, 36, 37, 39] supplies inputs clean enough that a single distance-two distillation step may suffice for some applications. The error rates that large algorithms demand [12], however, call for distance three or more, and there the landscape beyond $n \leq 54$ is uncharted.

Our method rests on one observation. Each input flips a set of checks, which we call its syndrome. When no two inputs touch the same wires, distance at least three is equivalent to the syndromes being nonzero and pairwise distinct [45] (see Sec. III A). We therefore choose the syndromes first, and we call the set of syndromes a factory uses its parent. For each parent, an enumeration of compatible output rows determines the factories it supports. Enumerating these rows for every parent

gives the exhaustive search used in our companion paper [41] to extend the classification of Ref. [40]. In the language of triorthogonal codes, the parent spans the X stabilizers and the output rows are the logical operators. When this is no longer feasible, we use symmetries to restrict the parent search. A group acting on the syndromes turns the choice of a parent into a choice of its orbits. This is our *symmetry slot ansatz*. The choice of group or parent to start from remains heuristic, and we delegate this choice to AI agents.

A *large language model* (LLM) is a neural network trained to continue text and an *agent* uses an LLM to perform multi-step tasks such as reading solver outcomes and writing proposals for later runs. An *orchestrator* [49–51] coordinates several agents working on one task. Agent-based methods have already had success in quantum error correction. Cruz-Benito et al. used an LLM-guided evolutionary search to discover bivariate bicycle codes, a family of quantum low-density parity-check codes [52]. Multi-agent systems have co-designed codes with transversal diagonal gates [53]. Other LLM-driven, reinforcement-learning and evolutionary systems have found codes, encoders and encodings [54–59]. Beyond quantum, program search with language models has produced new constructions in combinatorics and new algorithms [60, 61]. Neuro-symbolic systems pair a proposing model with an exact engine that checks every step [62]. In our search, the LLM proposes candidate structures and a deterministic verifier checks the resulting circuits.

Throughout, the protocols are *postselected diagonal CSS protocols*, that is, level-three diagonal rotations with independent Z faults, checks of pure X type, and rejection on any nonzero outcome. Every factory we report is an explicit circuit, available in our public database [63].

Section II reports the catalog and its highlights. Section III develops the matrix formulation and its two readings, as a code and as a circuit. Section IV describes the output search for a fixed parent, Sec. V introduces symmetry slots, and Sec. VI presents the agent workflow. Section VII discusses limitations, outlook and future directions.

II. RESULTS

This section reports what our searches found. Every class is a postselected diagonal CSS protocol in the sense of Sec. I, written $[[n, k, d]]$ for n inputs, k outputs and distance d . Catalytic and Hadamard-assisted protocols lie outside this class and could reach parameters the catalog does not [26, 45], so every comparison below is within the class.

Our searches produced 699 *classes*, a class being a quadruple $(n, k, d, \text{output gate})$ with the gate taken up to an invertible change of the output basis, the group $GL(k, 2)$, together with diagonal Clifford corrections (see Appendix D for the full list). In other words, a class

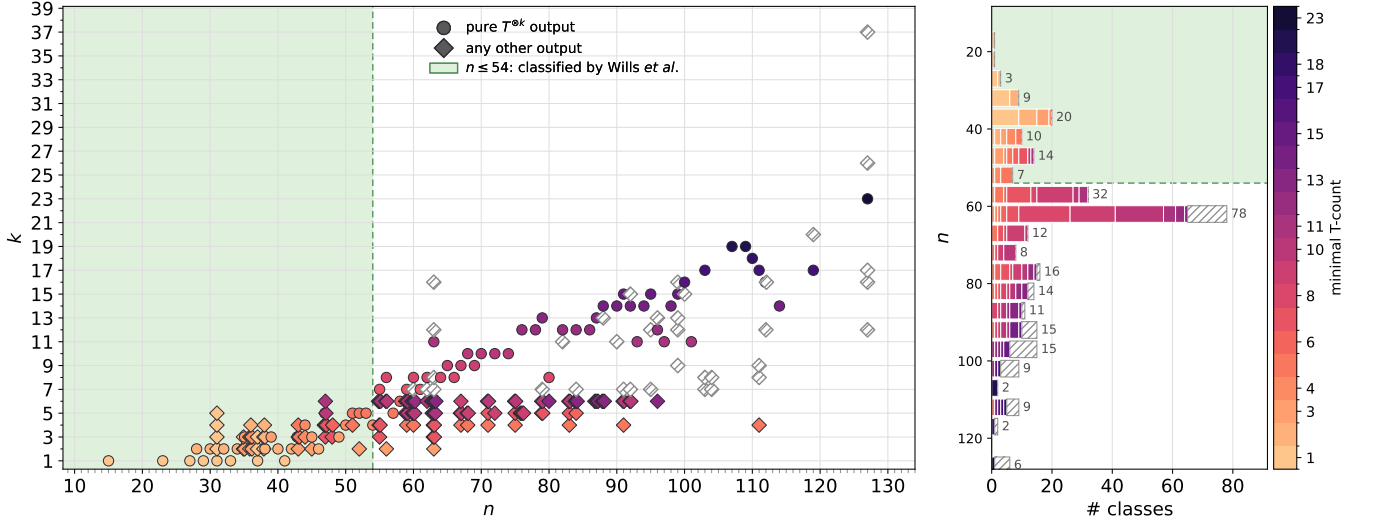


FIG. 1. The distance-three catalog up to $n = 127$. A circle is a factory with a pure- $T^{\otimes k}$ output, and a diamond one with any other output. Color is the T-count of the output gate, labeled “minimal T-count” on the color bar. Hatched markers are entangled outputs too wide for that exact minimization. A pure-T factory is omitted when a wider one at the same n and d already delivers it. The green band is $n \leq 54$, the range the companion paper [41] classifies exhaustively at this distance. The markers inside the band are classes that the methods of this paper found independently of the exhaustive classification. The right panel counts the drawn classes in bins of five consecutive n across the same range and splits each bin by T-count. The bins are placed so that $n = 54$ falls on a boundary, and no bar mixes the two sides of the band. Of the 294 classes drawn, 65 lie in the band, 225 are new in this work and the remaining four were already known.

TABLE I. The classes this paper reports, by distance, and how many of them are new. A class is a quadruple $(n, k, d, \text{output gate})$ as defined in the text. A class counts as new when no published protocol states it and it lies outside the range of exhaustive classifications.

d	3	4	5	6	7	8	9	all
classes	493	88	69	42	5	1	1	699
new	382	72	65	39	4	1	1	564

represents all the outputs connected by a CNOT+ S circuit. We identify 564 as new because they lie outside the classified ranges and published protocol parameters. The new classes span $n = 55$ to 1715, with 320 at $n \leq 127$.

To place outputs of different kinds on one scale, we use the minimum unitary T-count. It is the minimum number of T gates required in any circuit of CNOT and T gates to reach the output gate, computed exactly by Reed–Muller coset decoding [64]. We use this computation up to $k = 6$. The T-count of a pure- $T^{\otimes k}$ output is k , a CCZ output 7 and a CS output 3. We count over CNOT+T circuits because a native output replaces as many separately distilled T states as its T-count, the states that conventional synthesis would spend on that gate, the mechanism behind synthillation [23, 24]. This metric describes the synthesis cost within CNOT+T circuits. It does not determine a factory’s full implementation cost. Figures 1 and 2 draw the classes on the (n, k) plane, colored by this count. Table II lists selected fac-

TABLE II. Some highlighted factories resulting from our methods. The $[[850, 128, 6]]$ factory has overhead exponent $\gamma = 1.057$ (Table III), the lowest we know at $n \leq 1000$. Each factory listed has the smallest input count we know of for its output at its distance.

output gate	factory	N
$T^{\otimes 7}$	$[[55, 7, 3]]$	14
$T^{\otimes 11}$	$[[63, 11, 3]]$	17
$T^{\otimes 16}$	$[[100, 16, 3]]$	23
$T^{\otimes 23}$	$[[127, 23, 3]]$	30
$T^{\otimes 128}$	$[[850, 128, 6]]$	166
$CS^{\otimes 3}$	$[[63, 6, 3]]$	12
$CCZ^{\otimes 2}$	$[[63, 6, 3]]$	12
$CCZ^{\otimes 4}$	$[[100, 12, 4]]$	20
$T \otimes CCZ^{\otimes 2}$	$[[63, 7, 3]]$	13
$T^{\otimes 9} \otimes CCZ$	$[[63, 12, 3]]$	18
$T \otimes CCZ^{\otimes 4}$	$[[99, 13, 3]]$	20
$T^{\otimes 4} \otimes CCZ^{\otimes 4}$	$[[112, 16, 3]]$	23

ories which have the smallest input count we know of among postselected diagonal CSS protocols for its output gate at its stated distance.

Codes with transversal T gates reach $\gamma \rightarrow 0$ [31, 65], but only as the block length grows, and their performance at the lengths a machine could afford has not been established. We therefore compare only explicit constructions with $n \leq 1000$. The natural comparison for the

TABLE III. The punctured Reed-Muller factories tabulated by Haah and Hastings [20], beside the lowest overhead exponent we find at $n \leq 1000$ for the same distance. Every factory in the table is a pure $T^{\otimes k}$ factory, and $\gamma = \log(n/k)/\log d$ is the overhead exponent, where lower is better. At each distance, our work achieves the lowest exponents known and the distance-six one is the lowest known at any distance at $n \leq 1000$.

d	Ref. [20]		this work	
	factory	γ	factory	γ
3	[[863, 161, 3]]	1.528	[[750, 210, 3]]	1.159
4	[[872, 152, 4]]	1.260	[[429, 83, 4]]	1.185
5	[[887, 137, 5]]	1.161	[[879, 145, 5]]	1.120
6	[[912, 112, 6]]	1.170	[[850, 128, 6]]	1.057

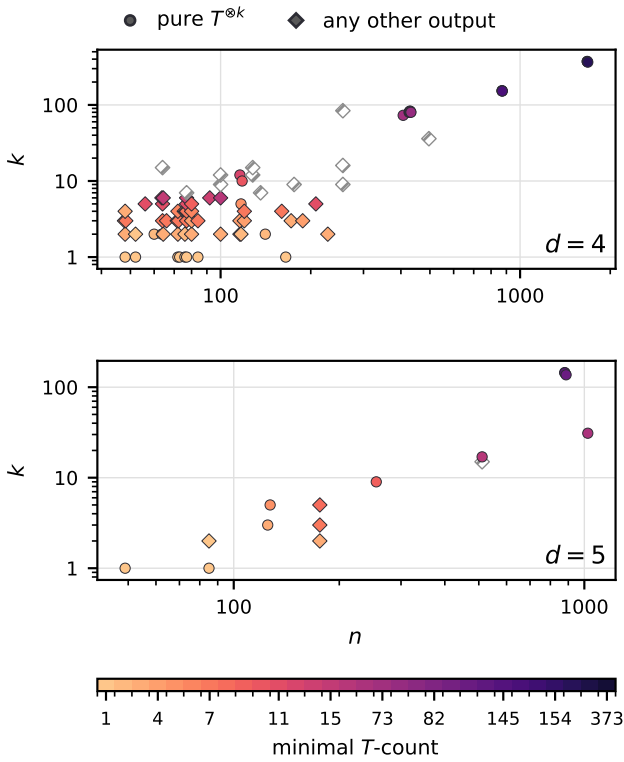


FIG. 2. The catalog at distances four and five, on logarithmic axes from $n = 48$ to 1683. Conventions are as in Fig. 1, with pure-T factories at any n represented by their highest k member. Of the 97 classes drawn, 78 are new in this work.

wide pure- $T^{\otimes k}$ end of the catalog is the punctured Reed-Muller triorthogonal family of Haah and Hastings [20], whose tables give explicit codes at each distance from three to seven. Table III compares their tabulated codes at distances three to six with our results at the same distance. Their best tabulated value, $\gamma = 1.161$, is achieved by [[887, 137, 5]]. Our searches reach a lower exponent at every distance in the table, and [[850, 128, 6]] at $\gamma = 1.057$ is the lowest we know of for an explicit pure-T factory

at $n \leq 1000$. We also find a [[1715, 287, 6]] factory at $\gamma = 0.998$, the smallest known pure-T factory with an exponent below one. Our entries were found by instantiating the frameworks of Refs. [31, 65] with the puncture searches of Appendix B.

The improvement is not confined to the largest codes. At distance three, the [[127, 23, 3]] factory uses approximately 5.52 inputs per output, compared with 5.74 for their [[109, 19, 3]]. Our [[63, 11, 3]] has a similar ratio, 5.73 inputs per output, while consuming 63 inputs rather than 109. Its exponent, $\gamma = 1.589$, is the lowest we know among pure-T factories with $n \leq 100$.

III. FACTORY AS A BINARY MATRIX

We represent a magic-state factory by a binary matrix whose columns specify its parity rotations. The matrix can then be read in two ways. Read as a code, it is a triorthogonal code or one of its generalizations [19, 20]. Read as a circuit, it is the list of parity rotations of a circuit construction [45, 46]. Each reading takes a different object as the unknown. We do not commit to either reading and work with the matrix itself. We choose its check part first and its outputs second, the syndrome-first formulation that Sec. III A introduces.

A. The matrix and the design problem

Fix $N = k + r$ wires, k outputs and r checks, all prepared in $|+\rangle$. Each nonzero $\alpha \in \mathbb{F}_2^N$, a length- N vector over the binary field $\mathbb{F}_2 = \{0, 1\}$, names a parity rotation

$$Z_\alpha = \exp\left[i\frac{\pi}{8}\left(I - \prod_{a:\alpha_a=1} Z_a\right)\right], \quad (2)$$

the weight- w phase rotation $Z^{\otimes w}(\pi/8)$ of Refs. [21, 45] with $w = \text{wt}(\alpha)$, the Hamming weight of α . It is a T rotation of the parity of the wires in the support of α . It costs one input, because a ladder of CNOTs moves a single-qubit T onto any parity (Fig. 3, legend). If that input is faulty, the same α is the Z fault it leaves behind. A *reduced circuit* is an $N \times n$ binary matrix G with distinct nonzero columns, one per input. Repeated columns cancel up to a Clifford, since two T rotations of the same parity compose to a Clifford rotation. Column m marks the wires that rotation m touches, and row operations are CNOTs. We call a factory *full rank* when its check rows are independent and its output rows are independent modulo the check span, equivalently when $\text{rank } G = N$. When the columns are the unknown we write the same matrix as G_y , where the selection $y \in \mathbb{F}_2^{2^N-1}$ has one bit per nonzero α and records which of the $2^N - 1$ possible columns appear.

The gate the circuit implements can be read off by counting columns. A parity rotation Z_α multiplies a computational basis state $|z\rangle$ by $e^{i\pi/4}$ raised to the parity $\bigoplus_{a:\alpha_a=1} z_a$. That parity obeys the integer iden-

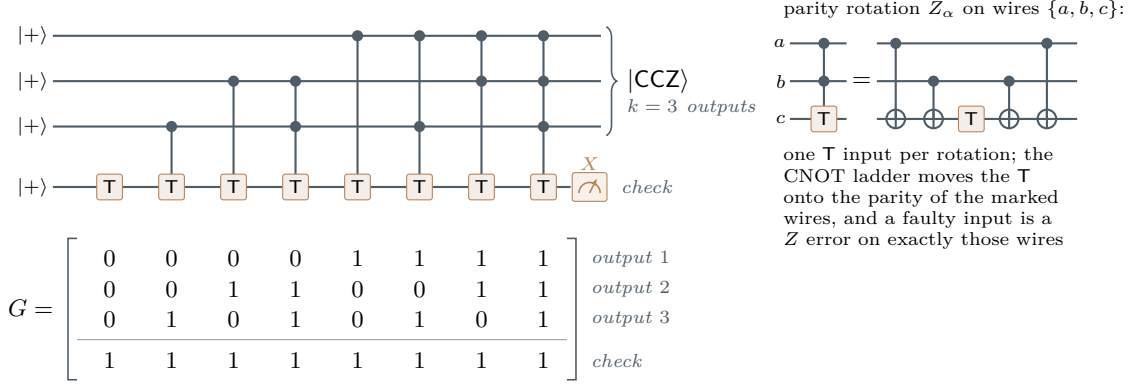


FIG. 3. The $[[8, 3, 2]]$ CCZ factory as a circuit [45] and as a matrix [66, 67]. Four wires prepared in $|+\rangle$ receive eight parity rotations Z_α , drawn as a vertical connector with a dot on each wire in the support of α and the T box on the check wire. Beneath each rotation is its corresponding column in the matrix G with support on every wire the rotation touches, the top three rows being the outputs and the bottom row the check. At the end the check wire is measured in the X basis and the run is kept only if the outcome is $+1$ (no check flipped). Three output wires carry the delivered $|\text{CCZ}\rangle = \text{CCZ}|+++\rangle$ state. By Eq. (3), every column count is even except that of the triple $\{1, 2, 3\}$, which lies only in $(111 | 1)$, so the circuit applies CCZ_{123} . A single fault flips the check and is detectable. Two faults cancel on the check but, being distinct columns, differ on the outputs, so $d = 2$ [45].

tity $\bigoplus_a z_a \equiv \sum_a z_a - 2\sum_{a<b} z_a z_b + 4\sum_{a<b<c} z_a z_b z_c \pmod{8}$. Applying all n rotations and collecting terms, the circuit acts, up to a diagonal Clifford, as

$$\prod_a \mathsf{T}_a^{c_a} \prod_{a<b} \mathsf{CS}_{ab}^{c_{ab}} \prod_{a<b<c} \mathsf{CCZ}_{abc}^{c_{abc}}, \quad (3)$$

where the *column counts* c_a , c_{ab} and c_{abc} are the numbers of columns that contain wire a , both a and b , and all three of a , b and c . The sign of the middle term is absorbed into the diagonal Clifford, since CS^{-1} differs from CS by a controlled- Z . Since T^2 , CS^2 and CCZ^2 are Clifford gates, only the parities of these counts carry non-Clifford content. A wire, pair or triple lying in an odd number of columns receives a T, a CS or a CCZ respectively, and the rest is a diagonal Clifford that is tracked classically. In the matrix G , wire a is row g_a , and the count c_Q of columns containing the wires in a set Q is the Hamming weight of the coordinatewise product $\bigodot_{a \in Q} g_a$, so the gate is fixed by the parities

$$\text{wt}\left(\bigodot_{a \in Q} g_a\right) \pmod{2}, \quad 1 \leq |Q| \leq 3. \quad (4)$$

We call this parity the *moment* of Q , and $|Q|$ its *degree*, so degrees one, two and three carry the T, CS and CCZ content. Figure 3 carries out this count for a worked example, the $[[8, 3, 2]]$ CCZ factory.

The gate is thus a diagonal gate in the third level of the Clifford hierarchy, given as a product of integer powers of T, CS and CCZ and determined up to a tracked diagonal Clifford gate. The target vector D lists the parities the output gate requires, one bit for each index Q with $|Q| \leq 3$. Every entry whose Q touches a check wire is zero, because a check must carry no non-Clifford phase or entanglement with the output wires. The monomial

matrix R_N , with entries $(R_N)_{Q,\alpha} = \prod_{a \in Q} \alpha_a$, has one row per index Q and one column per nonzero α , and its entry is one exactly when α contains Q . The product $R_N y$ therefore counts, modulo two, the selected columns that contain each Q . This is exactly the parity of Eq. (4). For the factory of Fig. 3, the target vector has a single one, at $Q = \{1, 2, 3\}$, with the rest being zero.

A selection y is a valid factory for the target D when $R_N y = D$. A valid factory still has to protect its outputs, and the distance d of Sec. I, which we now write $d(G)$, measures how well it does. Split each column into its output and check parts, $\alpha^m = (P_m | S_m) \in \mathbb{F}_2^k \times \mathbb{F}_2^r$. S_m is the *syndrome* of fault m , the checks it flips, and P_m is its *label*, the outputs it corrupts. A set F of faulty inputs is *accepted* when its syndromes cancel and *harmful* when its labels do not. The circuit distance is then

$$d(G) = \min_F \{|F| : \bigoplus_{m \in F} S_m = 0, \bigoplus_{m \in F} P_m \neq 0\}. \quad (5)$$

the size of the smallest fault set that is both accepted and harmful. The factory design problem is then a search for y such that

$$R_N y = D \quad \text{and} \quad d(G_y) \geq d. \quad (6)$$

Minimizing $n = \text{wt}(y)$ optimizes the cost.

The distance condition is simpler than it looks. For a reduced circuit, Eq. (5) gives $d(G) \geq 2$ if and only if every syndrome S_m is nonzero, since a zero syndrome makes its single fault accepted and, by reducedness, harmful. The distance-three condition is the observation this work rests on.

Theorem 1 (distance-three syndrome criterion). *For a reduced circuit G , $d(G) \geq 3$ if and only if its syndromes are nonzero and pairwise distinct.*

Proof. Nonzero syndromes exclude single faults. A pair i, j is accepted iff $S_i = S_j$, and it is then harmless only if $P_i = P_j$, which would make the two columns identical, contrary to reducedness. $\square$

Distance three is therefore a condition on the syndromes alone, and a distance- ≥ 3 circuit is specified by

$$\underbrace{\mathcal{S} \subseteq \mathbb{F}_2^r \setminus \{0\}}_{\text{which syndromes occur}} \quad \text{and} \quad \underbrace{\lambda : \mathcal{S} \rightarrow \mathbb{F}_2^k}_{\text{what each one outputs}}, \quad (7)$$

where the *parent* $\mathcal{S}$, the set of syndromes that occur, fixes what is detected, and the *labeling* λ , with $\lambda(S_m) = P_m$, fixes what is delivered. Since $n = |\mathcal{S}|$, a factory with r checks has at most $2^r - 1$ inputs. Higher distances are conditions of the same kind. For instance, $d \geq 4$ iff λ is additive on every triple of syndromes summing to zero. Writing a factory this way enforces distance three without enumerating fault sets, which is what makes our searches affordable.

B. Reading it as a code

Split the rows of G_y into the output block P , whose columns are the labels P_m , and the check block S , whose columns are the syndromes S_m and whose column set is the parent $\mathcal{S}$. Then $G_y = \begin{pmatrix} P \\ S \end{pmatrix}$ is the generator data of a CSS code. Its X stabilizers are the row space of S and its Z stabilizers are $G_y^\perp$, the vectors orthogonal to every row of G_y under the binary inner product $\langle u, v \rangle = \bigoplus_i u_i v_i$. The two commute because $\text{span } S \subseteq \text{span } G_y$. If $\text{rank } S = r$ and $\text{rank } G_y = r + k$, the code encodes k logical qubits. Row by row, $R_N y = D$ prescribes the moments of Eq. (4). For the target $T^{\otimes k}$ this is triorthogonality [19], and for other diagonal and entangled outputs its generalization [20, 23, 24].

The distance of Eq. (5) is the Z -distance of this code: a Z operator with support $e \in \mathbb{F}_2^n$ commutes with every X check when $Se = 0$ and is a stabilizer when also $Pe = 0$, so $d_Z = \min\{\text{wt}(e) : Se = 0, Pe \neq 0\}$, which is Eq. (5) with e for F . Only the Z -distance matters because input faults are Z faults and the inner code handles the rest. This reading connects factories to weight enumerators, puncturing, concatenation and the classification of small triorthogonal codes [40].

C. Reading it as a circuit

Section III A already builds G from a circuit, and Fig. 3 works through the $[[8, 3, 2]]$ example. Each row is a wire, each column is an input and the fault it can leave. A selection y implements the target when $R_N y = D$. The circuit reading therefore adds no new objects. It offers two perspectives on the same circuit, which lead to two ways of searching for a solution of Eq. (6). A *direct circuit search* looks for the selection y itself. A *borrowed-identity*

search starts from a circuit that acts as the identity and deletes columns from it.

The first perspective treats Eq. (6) as a *satisfiability* (SAT) problem, a set of Boolean variables together with constraints they must satisfy jointly, as in the circuit search of Jacinto et al. [46] and the solver searches for triorthogonal codes [47]. The variables are the bits of the selection y , one per candidate column. The target enters as linear equations in y , and the distance as clauses that forbid each low-weight fault set that is accepted and harmful. The cost lies in the distance clauses, which range over fault subsets, and Theorem 1 removes that cost up to $d = 3$.

The second perspective starts from circuits whose overall action is a diagonal Clifford. These are the selections h with $R_N h = 0$, which form the kernel $\mathcal{K}_N = \ker R_N$. A selection $h \in \mathcal{K}_N$ is an identity up to a diagonal Clifford, and *borrowed identities* [45] lie in this kernel. Any two solutions y, y' of $R_N y = D$ differ by an element of $\mathcal{K}_N$, so the solution set is one *coset* of the kernel, a copy of $\mathcal{K}_N$ shifted by a single solution y_0 ,

$$\{y : R_N y = D\} = y_0 + \mathcal{K}_N. \quad (8)$$

We call $h = y + y_0$ the *displacement* of a solution y from y_0 . The minimum input count for a target is the weight of a *coset leader*, the lightest element of that coset. Finding one is the decoding problem that Amy and Mosca identified with T-count minimization [64].

An identity becomes a factory by deletion. Pick a bare circuit b_D that implements the target on the k output wires alone, so that every column of b_D is *output-only*, with no support on a check wire. For any y_0 with $R_N y_0 = D$, the sum $h = y_0 + b_D$ lies in $\mathcal{K}_N$. Conversely, toggling the output-only columns b_D out of an identity h leaves

$$y_0 = h + b_D, \quad (9)$$

an encoded factory for D . This is the deletion step of Ref. [45]. The surviving columns all touch a check, so the distance is at least two by construction. The output is only as rich as the output-only columns the identity had to delete. A strict borrowed identity in the sense of Ref. [45] is a global phase on every basis state. In the column counts c_Q of Eq. (3), this requires

$$c_a \equiv 0 \pmod{8} \quad \text{for every wire } a, \quad (10)$$

$$c_{ab} \equiv 0 \pmod{4} \quad \text{for every pair } a < b, \quad (11)$$

$$c_{abc} \equiv 0 \pmod{2} \quad \text{for every triple } a < b < c. \quad (12)$$

Equation (9) needs only $R_N h = 0$, which permits a left-over diagonal Clifford and is therefore strictly more general.

The code reading and both perspectives of the circuit reading describe one object, and the following theorem says so. Item (i) is the circuit reading with the selection y as coordinates, item (ii) is the code reading, and item (iii) is the circuit reading with the displacement h as coordinates.

Theorem 2 (one coset, two readings). *For selections y, y' and any attainable level-three diagonal target D :*

- (i) y and y' implement the same gate up to a diagonal Clifford and global phase if and only if $R_N y = R_N y'$.
- (ii) For every $Q \subseteq \{1, \dots, N\}$ with $1 \leq |Q| \leq 3$, $(R_N y)_Q = \text{wt}(\bigodot_{a \in Q} g_a) \bmod 2$. Hence G_y is the generalized triorthogonal matrix whose prescribed moments are D .
- (iii) For any y_0 with $R_N y_0 = D$, the map $h \mapsto y_0 + h$ is a bijection from $\mathcal{K}_N$ onto $\{y : R_N y = D\}$.

Proof. By Eq. (3), two selections differ by a diagonal Clifford exactly when their column-count parities, the entries of $R_N y$ and $R_N y'$, agree, proving (i). For (ii), $\bigodot_{a \in Q} g_a$ is one in column m exactly when $\alpha_a^m = 1$ for all $a \in Q$, and its parity is the Q th entry of $R_N y$. For (iii), $R_N(y_0 + h) = D$ iff $R_N h = 0$, which is the definition of $\mathcal{K}_N$. $\square$

TABLE IV. The binary matrix formulation underlies both readings of a magic-state factory. Read as a code, the matrix is a triorthogonal code. Read as a circuit, it is searched through a borrowed identity or directly.

reading	perspective	search object
code	triorthogonal code	rows of G_y
circuit	borrowed identity	displacement $h \in \mathcal{K}_N$
	direct circuit search	selection y

A factory found by deleting from an identity and the “same” factory found by a triorthogonal solver are therefore the same point of the coset $y_0 + \mathcal{K}_N$ of Eq. (8). The two readings, and the two perspectives within the circuit reading, use different coordinates on that coset (Table IV). Imposing distance at least three makes available an intermediate between the two readings, the syndrome-first formulation of Eq. (7), which we use for the searches below.

IV. PARENT EVALUATION

For a fixed parent, the remaining variables are the output rows. The conditions for compatibility with the checks are linear or bilinear over $\mathbb{F}_2$. An output row must not leak a non-Clifford phase onto a check, two output rows may coexist only if their coordinatewise product is orthogonal to every check, and two rows that differ by a check row are the same output. The first and third conditions make the legal output directions the quotient

$$V(S) = (S^{(2)})^\perp / S, \quad (13)$$

and the second is the *compatibility condition*

$$\langle a \odot b, s \rangle = 0 \quad \text{for every } s \in S \quad (14)$$

on any two output representatives a, b . Here, $S^{(2)} = \text{span}\{s \odot s' : s, s' \in S\}$ is the Schur square of S , the span of all coordinatewise products of check rows. Since $s \odot s = s$ over $\mathbb{F}_2$ it contains S . The target condition of Sec. III A makes every overlap that involves a check row even, so $S \subseteq (S^{(2)})^\perp$. The dimension of $V(S)$ is the parent’s *capacity* κ , given by

$$\kappa = \dim V(S) \geq n - 2r - \binom{r}{2}. \quad (15)$$

Since $S^{(2)}$ is spanned by the r check rows and their $\binom{r}{2}$ pairwise products, its orthogonal complement has dimension at least $n - r - \binom{r}{2}$. The quotient by S removes at most r more. Within the space $V(S)$, one chooses a valid frame, that is, k elements $a_1, \dots, a_k$ of $V(S)$ that are independent and satisfy Eq. (14) in pairs. The output gate is then read off from the parities of the chosen frame via Eq. (4).

For a fixed parent, we compute $V(S)$ and enumerate compatible independent output frames. Each frame determines an output gate through its overlap parities. The dimension κ controls the size of the frame space. A completed enumeration gives all compatible gates for that parent, including product, controlled-phase and entangled outputs, without specifying a target gate in advance. For example, an incomplete enumeration of the parent that consists of all nonzero columns of length 6 already found 418,281 distance-three factories at $n = 63$, inequivalent up to a change of the output basis, the group $\text{GL}(k, 2)$. When full enumeration is infeasible, we can instead search for a specified target. Given a partial frame, the overlap conditions on the next output row form an affine system. Such a solve tests extensions of that partial frame. Ruling out the target requires exhausting the relevant partial frames.

V. SYMMETRY SLOTS

The parent evaluation of Sec. IV searches for compatible outputs once the syndromes are fixed. The remaining challenge is to find promising parents: with r checks, there are $2^{2^r - 1}$ subsets of nonzero syndromes to consider, already 2^{127} at $r = 7$. We use group symmetry to organize this search into families that can be explored efficiently.

Our *symmetry slot ansatz* groups syndromes into orbits under a chosen group action. We require the parent to be a union of these orbits and assign a common output label to all syndromes in each selected orbit. The solver then chooses the orbits and their labels jointly to realize the target gate. This replaces decisions about individual syndromes with decisions about entire orbits, reducing the search space while allowing both the parent and its output labeling to be determined within the chosen symmetry.

Let $\mathcal{G} \leq \text{GL}(r, 2)$, the group of invertible $r \times r$ binary matrices, act on the nonzero syndromes. Let $\Omega_{\mathcal{G}}$ be the

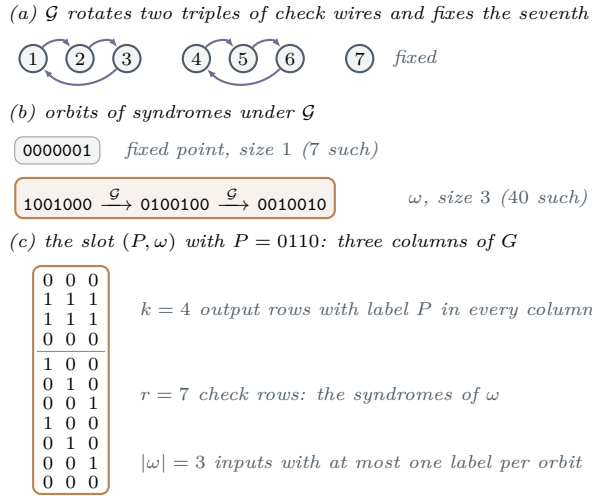


FIG. 4. The slot ansatz, drawn for one example group. (a) $\mathcal{G}$ is the order-three permutation of the $r = 7$ check wires that rotates $\{1, 2, 3\}$ and $\{4, 5, 6\}$ and fixes wire 7. (b) Syndromes fall into orbits under $\mathcal{G}$. The seven syndromes constant on each triple are fixed points, and the other 120 form 40 orbits of size three, 47 orbits in all. (c) A slot pairs an orbit ω with an output label $P \in \mathbb{F}_2^k$ and stands for the columns $(P | s)$ of G , one for each $s \in \omega$.

set of orbits, the equivalence classes of syndromes under $\mathcal{G}$, with $q_{\mathcal{G}} = |\Omega_{\mathcal{G}}|$. A *slot* is a pair (P, ω) with $\omega \in \Omega_{\mathcal{G}}$ and $P \in \mathbb{F}_2^k$. Selecting the slot (P, ω) places in G one column $(P | s)$ for each $s \in \omega$, so all of them carry the same label P . Each orbit may be assigned at most one output label. A circuit matrix G is then specified by a selection of slots whose columns together realize the target gate, $R_N y = D$. This is Eq. (7) with orbits in place of syndromes, and the input count is

$$n = \sum_{(P, \omega) \text{ selected}} |\omega|. \quad (16)$$

Figure 4 illustrates the ansatz for a representation of the cyclic group C_3 .

Orbits are disjoint and each carries at most one label, so the selected columns have distinct nonzero syndromes and thus, by Theorem 1, distance 3 is built-in. The ansatz is therefore a SAT problem. One Boolean variable per candidate slot records whether that slot is selected. There are at most $2^k q_{\mathcal{G}}$ variables instead of $2^k(2^r - 1)$, one at-most-one constraint per orbit, and parity constraints on orbit sums for each index Q of the target D . We pass the instance to solvers such as CP-SAT or CryptoMiniSat. A solver may find a solution, establish infeasibility, or stop without resolving the instance. For trivial $\mathcal{G}$ every orbit is a singleton and the instance is the unrestricted one over all reduced distance- ≥ 3 factories with r checks. A nontrivial $\mathcal{G}$ compresses the search but narrows its scope.

The group of Fig. 4, an order-three permutation that rotates two triples of check wires and fixes the rest, shows

the size of the compression. At $r = 7$ it has $q_{\mathcal{G}} = 47$ orbits, seven fixed points and forty of size three, so at $k = 4$ the SAT instance carries $2^4 \times 47 = 752$ slot variables where the unrestricted instance carries $2^4 \times 127 = 2032$. The parent count shrinks as well. A parent is now a choice of orbits rather than of syndromes, giving 2^{47} candidates instead of 2^{127} , a reduction by a factor of 2^{80} . On six check wires the same group leaves 23 orbits, 368 slot variables against 1008, and 2^{23} parents against 2^{63} . Among the factories of Sec. II, the slot search with this symmetry found $[[28, 2, 3]]$, $[[44, 4, 3]]$, $[[51, 5, 3]]$ and $[[85, 1, 5]]$ for pure $T^{\otimes k}$, $[[47, 4, 3]]$, and the $[[63, 4, 3]]$ $CS^{\otimes 2}$ factory.

VI. AGENT-DRIVEN SEARCH

Choosing groups by hand gives factories beyond the classified range, but little guidance on which groups to try next. We therefore use LLM agents to select groups, parents and puncture sets based on earlier search outcomes. Each search is organized as a *campaign*, a sequence of *rounds*. In each round, *proposer* agents write proposals from a fixed *brief*, a deterministic *engine* runs the solver each proposal names, an independent *verifier* checks every returned circuit, and a *reviewer* agent reads the outcomes and writes the *direction*, a short instruction for the next round.

A proposal names one of three kinds of object. The first is a *group* with its action on the check wires and a column count, which the slot solver of Sec. V converts into an orbit search. The second is a parent, a syndrome set whose compatible outputs are enumerated by the parent evaluation of Sec. IV. The third is a *puncture set*. Puncturing starts from one fixed code on many coordinates and deletes some of them. A deleted coordinate stops being an input and becomes an output, so deleting j independent coordinates from a code of length ℓ leaves a factory with $\ell - j$ inputs and j additional outputs [20]. This is how the largest factories in the catalog were found, where our searches cannot enumerate parents and labelings. It is also where the lowest overhead exponents of Table III come from, obtained by instantiating the frameworks of Refs. [31, 65]. Appendix B describes these searches in detail, and Appendix A gives the pruning rule used in the puncture search and two extension moves that build new factories from verified ones. A proposal may also be an explicit circuit, which bypasses the engine and goes straight to the verifier.

Each campaign consists of successive rounds of proposals and solver runs (Fig. 5). The proposers work from a fixed brief describing the factory representation, accepted proposal formats and search objective. Each round adds a direction written by the reviewer from the previous round's results. Both proposers and reviewers contribute to the proposal batch. An *ingest* step removes malformed and duplicate proposals, and the engine processes the remaining candidates. An independent verifier checks each

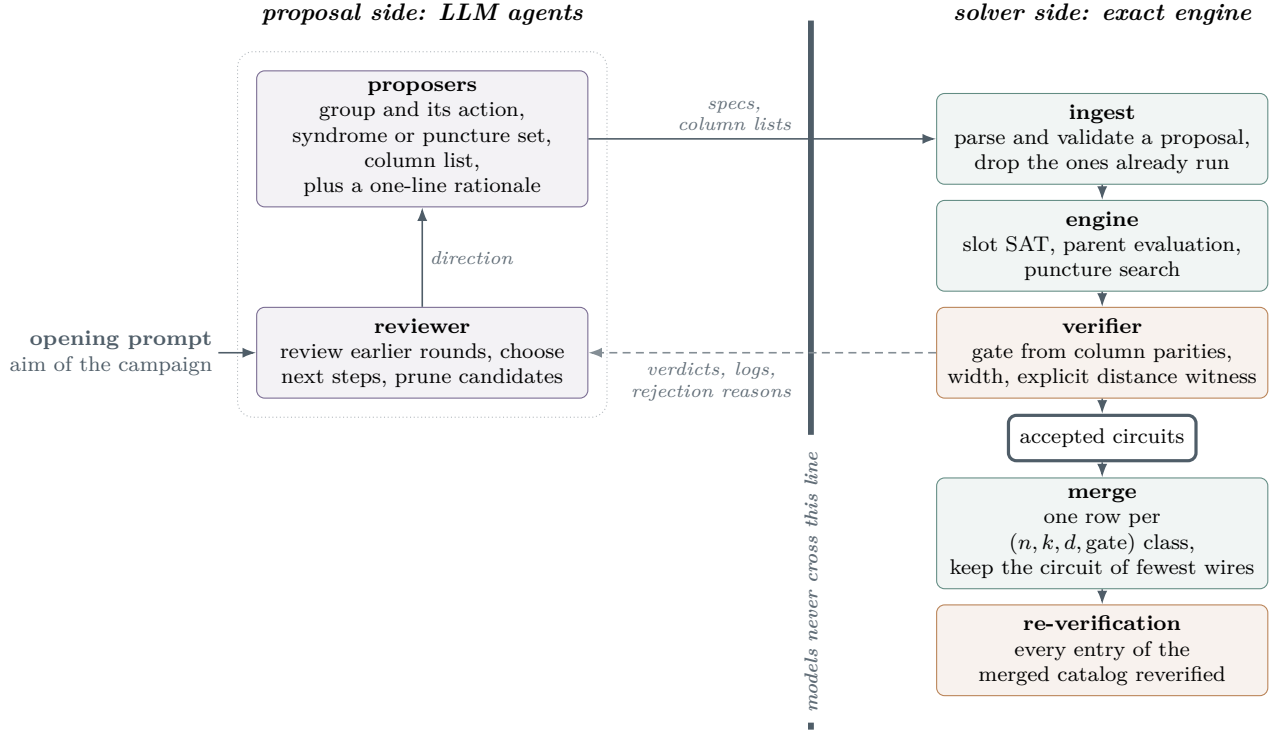


FIG. 5. The agent loop. An opening prompt, written by the user, sets the aim of the campaign. On the proposal side the reviewer reads the results of earlier rounds and sets a direction, which the proposers use to generate proposals. A proposal names a symmetry group and its action, a syndrome or puncture set, or an explicit column list, and carries a one-line rationale. The heavy line separates the two sides, and only files cross it. Proposals go right as specification files and column lists, and verdicts, logs and rejection reasons come back left. No model runs on the solver side. Ingest parses each proposal, rejects one it cannot read and drops one already run. The engine runs the tool the proposal names. Many proposals return no circuit, and those negatives are passed back to the proposal side along with the successes. A circuit that does come back goes to the verifier, which re-derives its gate, width and distance from the matrix alone, with no knowledge of the agents' proposal. Merge keeps one row per class, a class being $(n, k, d, \text{output gate})$ with the gate taken up to $GL(k, 2)$ and diagonal Clifford corrections, as in Sec. II. Where several circuits reach the same class, the one on fewest wires is kept. The merged catalog is reverified once more at the end of the round. One round is one pass around the loop, and a campaign is a sequence of rounds.

returned circuit before it enters the catalog. The catalog is then reverified as a whole. The reviewer reads the solver outcomes and verification logs to set the next direction. A campaign ends after its assigned number of rounds or when stopped by the authors. Appendix C follows one logged round from its direction to the next.

The verifier re-derives the target gate and check parities from the returned columns. It proves the claimed distance bound and checks any witness used to claim an exact distance. It also checks that the output rows are independent modulo the check span and that, in the reported output basis, every declared output occurs in the recovered non-Clifford gate. These checks exclude dependent logical labels and explicit spectator outputs.

Unsuccessful searches also guide the reviewer's next steps. A solver result of UNSAT rules out a solution within the specified constraints, whereas a timeout leaves the instance unresolved and may prompt a change in strategy or computational budget. In the puncture campaign, for example, repeated randomized restarts timed out without a conclusive result. The reviewer responded by fo-

cusings the search on a smaller neighborhood, where the solver completed the search and found an improved factory.

Across campaigns, we varied the number of reviewers and proposers, the models used, and the number of rounds. The full catalog of results is available in the public database [63].

VII. OUTLOOK

This work brings together a unified binary-matrix formulation, symmetry-based searches and LLM agents to discover magic-state factories beyond the reach of exhaustive classification. That classification now reaches $n \leq 54$ at distance three and above, at a cost of order 10^5 core-hours, and the same method is estimated to need 10^8 core-hours for the next input count [41]. We extend the search beyond this limit by combining symmetry reductions with LLM agents that use previous search outcomes to propose promising groups, parents and puncture sets.

These targeted searches yield both pure-T factories and entangled outputs built from combinations of T, CS and CCZ gates. The resulting catalog is a starting point. Our methods can readily be used to discover more factories by varying the search goals and constraints.

An immediate next step is to translate the improvements in input count and overhead exponent into savings in a fault-tolerant architecture. This requires accounting for acceptance probabilities, leading error coefficients and the Clifford circuits that consume the outputs. For entangled outputs, the advantage of distilling a desired gate directly also depends on the gate mix of the compiled algorithm. The explicit circuits in our catalog provide a basis for these comparisons and for searches guided by the cost of the complete computation.

The framework also offers several directions for further exploration. Within the present class of postselected diagonal protocols, searches at distance four and above can incorporate additional constraints on the output labels (Sec. III A). Extending the output gates to level four, including CCCZ, requires $\pi/16$ rotations and phase polynomials of degree four [18]. Broadening the search to catalytic and Hadamard-assisted protocols could yield further improvements [26, 45], with verifiers adapted to check these circuits directly. Across these extensions, the same workflow can be retained: agents guide the search, deterministic solvers construct candidates, and independent verifiers establish their properties.

To support continued discovery, we release the catalog, explicit circuits, search engine and verifiers together with a *context directory* for the quantum community [63]. The directory brings together the mathematical framework, search techniques and lessons from previous campaigns, including established results and open directions. These resources enable others to brief their own agents, build on our searches and adapt them to their gate mixes and resource requirements. We invite the community to extend these methods and contribute verified circuits, building an active, open-source repository that grows with each new effort.

USE OF AI

The framework, the search designs, the solver, the verifiers and the acceptance criteria in this work are the

authors' own. Within that framework, large language models were the proposing half of the loop of Sec. VI: they proposed parents, symmetry groups and search directions and, in the later runs, pruned candidates as reviewers. They also assisted with the scripts around the loop, the catalog build and the figure tooling. Every circuit here was re-derived from its stored columns by the verifiers of Sec. VI before entering the catalog. The authors are responsible for the results and claims in this paper.

The agents were Anthropic Claude models throughout. Reviewers were typically the latest available Fable or Opus versions, and proposers were Opus or Sonnet models. Some campaigns were run with HUMANIZE2 [68], an open-source orchestrator for workflows of coding agents, in a loop of the same shape as the one described in Sec. VI.

DATA AVAILABILITY

The catalog, the engine, the verifiers, the context directory of briefs and campaign notes, and every circuit in this paper are available in the public database of Ref. [63].

ACKNOWLEDGMENTS

We thank Anqi Gong, Andrew Cross, Juan-Cruz Benito, David Kremer and Swanand Ravindra Kadhe for helpful discussions. A.W. acknowledges support from the MIT-IBM Watson AI Lab. This preprint is assigned number MIT-CTP/6112. The theoretical framework, the symmetric search design and the acceptance criterias are the authors' own. We acknowledge the use of Anthropic AI models for help in building the code and generating proposals in our agent loop. Full details of the AI usage can be found in our section titled "Use of AI".

-
- [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th ed. (Cambridge university press, Cambridge, 2010).
 - [2] D. Gottesman, *Surviving as a Quantum Computer in a Classical World* (2026).
 - [3] D. J. Spencer, S. P. Jain, A. Tanggara, Z. Sun, T. Haug, D. Khu, and K. Bharti, *Quantum error correction and fault tolerance: A comprehensive tutorial* (2026), [arXiv:2605.29137 \[quant-ph\]](#).
 - [4] D. Gottesman, *Stabilizer Codes and Quantum Error Correction* (1997), [arXiv:quant-ph/9705052](#).
 - [5] A. G. Fowler and C. Gidney, *Low overhead quantum computation using lattice surgery* (2019), [arXiv:1808.06709 \[quant-ph\]](#).
 - [6] D. Litinski, A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery, *Quantum* **3**, 128 (2019).

- [7] B. Eastin and E. Knill, Restrictions on Transversal Encoded Quantum Gate Sets, *Physical Review Letters* **102**, 110502 (2009).
- [8] D. Gottesman and I. L. Chuang, Demonstrating the viability of universal quantum computation using teleportation and single-qubit operations, *Nature* **402**, 390 (1999).
- [9] X. Zhou, D. W. Leung, and I. L. Chuang, Methodology for quantum logic gate construction, *Physical Review A* **62**, 052316 (2000).
- [10] E. Knill, Quantum computing with realistically noisy devices, *Nature* **434**, 39 (2005).
- [11] S. Bravyi and A. Kitaev, Universal quantum computation with ideal Clifford gates and noisy ancillas, *Physical Review A* **71**, 022316 (2005).
- [12] C. Gidney, How to factor 2048 bit RSA integers with less than a million noisy qubits (2025), [arXiv:2505.15917 \[quant-ph\]](#).
- [13] H. Zhou, C. Duckering, C. Zhao, D. Bluvstein, M. Cain, A. Kubica, S.-T. Wang, and M. D. Lukin, Resource Analysis of Low-Overhead Transversal Architectures for Reconfigurable Atom Arrays, in *Proceedings of the 52nd Annual International Symposium on Computer Architecture* (2025) pp. 1432–1448.
- [14] Y. Li, A magic state’s fidelity can be superior to the operations that created it, *New Journal of Physics* **17**, 023037 (2015).
- [15] S. Singh, A. S. Darmawan, B. J. Brown, and S. Puri, High-Fidelity Magic-State Preparation with a Biased-Noise Architecture, *Physical Review A* **105**, 052410 (2022).
- [16] G. Zhang, Y. Zhu, X. Yuan, and Y. Li, Constant-Overhead Magic State Injection into qLDPC Codes with Error Independence Guarantees (2025), [arXiv:2505.06981 \[quant-ph\]](#).
- [17] K. Liu, S. Xu, T. Jochym-O’Connor, Z. He, S. Singh, and Y. Ding, In-Situ Simultaneous Magic State Injection on Arbitrary CSS qLDPC Codes (2026), [arXiv:2604.05126 \[quant-ph\]](#).
- [18] S. X. Cui, D. Gottesman, and A. Krishna, Diagonal gates in the Clifford hierarchy, *Physical Review A* **95**, 012329 (2017).
- [19] S. Bravyi and J. Haah, Magic state distillation with low overhead, *Physical Review A* **86**, 052329 (2012).
- [20] J. Haah and M. B. Hastings, Codes and Protocols for Distilling T , controlled S , and Toffoli Gates, *Quantum* **2**, 71 (2018).
- [21] C. Jones, Low-overhead constructions for the fault-tolerant Toffoli gate, *Physical Review A* **87**, 022328 (2013).
- [22] B. Eastin, Distilling one-qubit magic states into Toffoli states, *Physical Review A* **87**, 032321 (2013).
- [23] E. T. Campbell and M. Howard, Unified framework for magic state distillation and multiqubit gate synthesis with reduced resource cost, *Physical Review A* **95**, 022316 (2017).
- [24] E. T. Campbell and M. Howard, Unifying Gate Synthesis and Magic State Distillation, *Physical Review Letters* **118**, 060501 (2017).
- [25] E. T. Campbell, Catalysis and activation of magic states in fault-tolerant architectures, *Physical Review A* **83**, 032317 (2011).
- [26] C. Gidney and A. G. Fowler, Efficient magic state factories with a catalyzed $|CCZ\rangle$ to $2|T\rangle$ transformation, *Quantum* **3**, 135 (2019).
- [27] A. Gong, C. A. Pattison, P. Rall, and A. Wills, Magic State Distillation via Codes over Binary Extension Fields (2026), [arXiv:2608.09727 \[quant-ph\]](#).
- [28] A. Wills, M.-H. Hsieh, and H. Yamasaki, Constant-overhead magic state distillation, *Nature Physics* **21**, 1842 (2025).
- [29] L. Golowich and V. Guruswami, Asymptotically Good Quantum Codes with Transversal Non-Clifford Gates, in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, STOC ’25 (Association for Computing Machinery, New York, NY, USA, 2025) pp. 707–717.
- [30] Q. T. Nguyen, Good Binary Quantum Codes with Transversal CCZ Gate, in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, STOC ’25 (Association for Computing Machinery, New York, NY, USA, 2025) pp. 697–706.
- [31] R. San-José, Asymptotically good binary triorthogonal codes and higher-level transversal gates (2026), [arXiv:2609.08203 \[cs.IT\]](#).
- [32] D. Litinski, Magic State Distillation: Not as Costly as You Think, *Quantum* **3**, 205 (2019).
- [33] T. Itogawa, Y. Takada, Y. Hirano, and K. Fujii, Efficient Magic State Distillation by Zero-Level Distillation, *PRX Quantum* **6**, 020356 (2025).
- [34] C. Gidney, N. Shutty, and C. Jones, Magic state cultivation: Growing T states as cheap as CNOT gates (2024), [arXiv:2409.17595 \[quant-ph\]](#).
- [35] S. Jacoby, Y. Vaknin, A. Retzker, and A. L. Grimsmo, Magic State Injection with Erasure Qubits, *PRX Quantum* **6**, 040323 (2025).
- [36] K. Sahay, P.-K. Tsai, K. K. Chang, Q. Su, T. B. Smith, S. Singh, and S. Puri, Fold-transversal surface code cultivation, *PRX Quantum* **7**, 033006 (2026).
- [37] E. Rosenfeld, C. Gidney, G. Roberts, A. Morvan, N. Lacroix, D. Kafri, J. Marshall, M. Li, V. Sivak, D. Abanin, A. Abbas, R. Acharya, L. A. Beni, G. Aigeldinger, R. Alcaraz, S. Alcaraz, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, W. Askew, N. As-trakhantsev, J. Atalaya, R. Babbush, B. Ballard, J. C. Bardin, H. Bates, A. Bengtsson, M. B. Karimi, A. Bilmes, S. Bilodeau, F. Borjans, J. Bovaird, D. Bowers, L. Brill, P. Brooks, M. Broughton, D. A. Browne, B. Buchea, B. B. Buckley, T. Burger, B. Burkett, N. Bushnell, J. Busnaina, A. Cabrera, J. Campero, H.-S. Chang, S. Chen, Z. Chen, B. Chiaro, L.-Y. Chih, A. Y. Cleland, B. Cochrane, M. Cockrell, J. Cogan, P. Conner, H. Cook, R. G. Cortiñas, W. Courtney, A. L. Crook, B. Curtin, M. Damyanov, S. Das, D. M. Debroy, S. Demura, P. Donohoe, I. Drozdov, A. Dunsworth, V. Ehimhen, A. Eickbusch, A. M. Elbag, L. Ella, M. Elzouka, D. Enriquez, C. Erickson, L. Faoro, V. S. Ferreira, M. Flores, L. F. Burgos, S. Fontes, E. Forati, J. Ford, B. Foxen, M. Fukami, A. W. L. Fung, L. Fuste, S. Ganjam, G. Garcia, C. Garrick, R. Gasca, H. Gehring, R. Geiger, É. Genois, W. Giang, D. Gilboa, J. E. Goeders, E. C. Gonzales, R. Gosula, S. J. de Graaf, A. G. Dau, D. Graumann, J. Grebel, A. Greene, J. A. Gross, J. Guerrero, L. L. Guevel, T. Ha, S. Habegger, T. Hadick, A. Hadjikhani, M. C. Hamilton, M. Hansen, M. P. Harrigan, S. D. Harrington, J. Hartshorn, S. Heslin, P. Heu, O. Higgott, R. Hiltermann, J. Hilton, H.-Y. Huang, M. Hucka, C. Hudspeth, A. Huff, W. J. Huggins, L. B. Ioffe, E. Jeffrey, S. Jevons, Z. Jiang, X. Jin, C. Joshi, P. Juhas, A. Kabel, H. Kang, K. Kang, A. H. Karam-

- lou, R. Kaufman, K. Kechedzhi, T. Khatrar, M. Khezri, S. Kim, P. V. Klimov, C. M. Knaut, B. Kobrin, A. N. Korotkov, F. Kostritsa, J. M. Kreikebaum, R. Kudo, B. Kueffler, A. Kumar, V. D. Kurilovich, V. Kutsko, T. Lange-Dei, B. W. Langley, P. Laptev, K.-M. Lau, E. Leavell, J. Ledford, J. Lee, K. Lee, B. J. Lester, W. Leung, L. Li, W. Y. Li, A. T. Lill, W. P. Livingston, M. T. Lloyd, A. Locharla, L. D. Lorenzo, E. Lucero, D. Lundahl, A. Lunt, S. Madhuk, A. Maiti, A. Maloney, S. Mandrà, L. S. Martin, O. Martin, E. Mascot, P. M. Das, D. Maslov, M. Mathews, C. Maxfield, J. R. McClean, M. McEwen, S. Meeks, A. Megrant, K. C. Miao, Z. K. Minev, R. Molavi, S. Molina, S. Montazeri, C. Neill, M. Newman, A. Nguyen, M. Nguyen, C.-H. Ni, M. Y. Niu, N. Noll, L. Oas, W. D. Oliver, R. Orosco, K. Ottosson, A. Pagano, A. D. Paolo, S. Peek, D. Peterson, A. Pizzuto, E. Portoles, R. Potter, O. Pritchard, M. Qian, C. Quintana, G. Ramachandran, A. Ranadive, M. J. Reagor, R. Resnick, D. M. Rhodes, D. Riley, R. Rodriguez, E. Ropes, L. B. D. Rose, E. Rosenberg, D. Rosenstock, E. Rossi, P. Roushan, D. A. Rower, R. Salazar, K. Sankaragomathi, M. C. Sarihan, M. Schaefer, S. Schroeder, H. F. Schurkus, A. Shalingohar, M. J. Shearn, A. Shorter, N. Shutt, V. Shvarts, S. Small, W. C. Smith, D. A. Sobel, B. Spells, S. Springer, G. Sterling, J. Suchard, A. Szasz, A. Szein, M. Taylor, J. P. Thiruraman, D. Thor, D. Timucin, E. Tomita, A. Torres, M. M. Torunbalci, H. Tran, A. Vaishnav, J. Vargas, S. Vdovichev, G. Vidal, B. Villalonga, C. V. Heidweiller, M. Voorhees, S. Waltman, J. Waltz, S. X. Wang, D. Wang, B. Ware, J. D. Watson, Y. Wei, T. Weidel, T. White, K. Wong, B. W. K. Woo, C. J. Wood, M. Woodson, C. Xing, Z. J. Yao, P. Yeh, B. Ying, J. Yoo, N. Yosri, E. Young, G. Young, A. Zalcman, R. Zhang, Y. Zhang, N. Zhu, N. Zobrist, Z. Zou, H. Neven, S. Boixo, C. Jones, J. Kelly, A. Bourassa, and K. J. Satzinger, [Magic state cultivation on a superconducting quantum processor](#) (2025), [arXiv:2512.13908 \[quant-ph\]](#).
- [38] R. S. Gupta, N. Sundaresan, T. Alexander, C. J. Wood, S. T. Merkel, M. B. Healy, M. Hillenbrand, T. Jochym-O'Connor, J. R. Wootton, T. J. Yoder, A. W. Cross, M. Takita, and B. J. Brown, Encoding a magic state with beyond break-even fidelity, *Nature* **625**, 259 (2024).
- [39] Y. Hirano, R. Toshio, T. Itogawa, and K. Fujii, [Efficient magic state cultivation with lattice surgery](#) (2026), [arXiv:2510.24615 \[quant-ph\]](#).
- [40] S. Nezami and J. Haah, Classification of small triorthogonal codes, *Physical Review A* **106**, 012437 (2022).
- [41] A. Wills, S. P. Jain, and S. Singh, [Classification of Generalised Triorthogonal Codes through Length 54](#) (2026), [arXiv:2609.30860 \[quant-ph\]](#).
- [42] S. Bravyi and A. Cross, [Doubled Color Codes](#) (2015), [arXiv:1509.03239 \[quant-ph\]](#).
- [43] S. P. Jain and V. V. Albert, Transversal Clifford and T-gate codes of short length and high distance, *IEEE Journal on Selected Areas in Information Theory* **6**, 127 (2025).
- [44] M. Sullivan, Code conversion with the quantum Golay code for a universal transversal gate set, *Physical Review A* **109**, 042416 (2024).
- [45] S. Singh, C. Gidney, and C. Jones, [Borrowed Identities: Malleable Distillation Factories and a Unified Numerical Search](#) (2026), [arXiv:2606.28518 \[quant-ph\]](#).
- [46] H. Jacinto, X. Valcarce, V. Barizien, É. Gouzien, and N. Sangouard, [Exploring the landscape of compact magic-state distillation factories](#) (2026), [arXiv:2606.07734 \[quant-ph\]](#).
- [47] A. Baldelli, O. Å. Mostad, H.-Y. Lin, E. Rosnes, and M. Battagioni, [On Constructing and Decoding Quantum Triorthogonal Codes](#) (2026), [arXiv:2605.24519 \[quant-ph\]](#).
- [48] S. Singh, [Github Repository, Magic State Factory Search](#) (2026).
- [49] A. Adimulam, R. Gupta, and S. Kumar, [The Orchestration of Multi-Agent Systems: Architectures, Protocols, and Enterprise Adoption](#) (2026), [arXiv:2601.13671 \[cs.MA\]](#).
- [50] A. Roman and J. Roman, [Orchestral AI: A Framework for Agent Orchestration](#) (2026), [arXiv:2601.02577 \[cs.AI\]](#).
- [51] Y. Song, Y. Song, T. Pfister, and J. Yoon, [PaperOrchestra: A Multi-Agent Framework for Automated AI Research Paper Writing](#) (2026), [arXiv:2604.05018 \[cs.AI\]](#).
- [52] J. Cruz-Benito, A. W. Cross, D. Kremer, and I. Faro, [Evolutionary Discovery of Bivariate Bicycle Codes with LLM-Guided Search](#) (2026), [arXiv:2606.02418 \[quant-ph\]](#).
- [53] X. He, S. Lu, and B. Zeng, [Co-Designing Quantum Codes with Transversal Diagonal Gates via Multi-Agent Systems](#) (2026), [arXiv:2510.20728 \[quant-ph\]](#).
- [54] Z. Liu and F. Marquardt, [Large-Language-Model Discovery of Quantum LDPC Codes through Structured Concept Evolution](#) (2026), [arXiv:2606.24808 \[quant-ph\]](#).
- [55] G. Yan, S. Li, P. Ma, Q. Zhang, P. Ma, J. Wang, M.-H. Hsieh, and Y. Du, [OmniQEC: Discovering practical quantum error-correcting codes by an AI scientist](#) (2026), [arXiv:2607.25865 \[quant-ph\]](#).
- [56] K. Heitritter, J. Brown, and T. Hardikar, [Evolving Quantum Error-Correcting Encodings for Molecular Simulation](#) (2026), [arXiv:2606.25870 \[quant-ph\]](#).
- [57] H. P. Nautrup, N. Delfosse, V. Dunjko, H. J. Briegel, and N. Friis, Optimizing Quantum Error Correction Codes with Reinforcement Learning, *Quantum* **3**, 215 (2019).
- [58] J. Olle, R. Zen, M. Puviani, and F. Marquardt, Simultaneous discovery of quantum error correction codes and encoders with a noise-aware reinforcement learning agent, *npj Quantum Information* **10**, 126 (2024).
- [59] M. A. Webster and D. E. Browne, Engineering Quantum Error Correction Codes Using Evolutionary Algorithms, *IEEE Transactions on Quantum Engineering* **6**, 1 (2025).
- [60] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, Mathematical discoveries from program search with large language models, *Nature* **625**, 468 (2024).
- [61] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog, [AlphaEvolve: A coding agent for scientific and algorithmic discovery](#) (2025), [arXiv:2506.13131 \[cs.AI\]](#).
- [62] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, Solving olympiad geometry without human demonstrations, *Nature* **625**, 476 (2024).
- [63] S. Jain and S. Singh, [GitHub Repository, Magic State Factories Repo](#) (2026).

- [64] M. Amy and M. Mosca, T-Count Optimization and Reed–Muller Codes, *IEEE Transactions on Information Theory* **65**, 4771 (2019).
- [65] A. Wills, Improved Quantum Codes with Transversal T Gates (2026), [arXiv:2608.24000 \[quant-ph\]](#).
- [66] A. Kubica, B. Yoshida, and F. Pastawski, Unfolding the color code, *New Journal of Physics* **17**, 083026 (2015).
- [67] E. T. Campbell, The smallest interesting colour code (2016).
- [68] Z. Zhang and The humanize contributors, *Humanize* (2026).

Appendix A: Pruning and extension rules used in the search

We prove one pruning rule and two extension moves. Downward closure gives the pruning rule used in the RM(3, 10) search of Appendix B. Free-row appending was used after puncturing the seven-check simplex parent, which contains all 127 nonzero columns in $\mathbb{F}_2^7$. One resulting factory is $[[112, 16, 3]]$. Parity lift gives another certified extension and maps $[[99, 12, 3]]$ to $[[100, 12, 4]]$.

For the puncture search, write g_x for the column of degree- ≤ 3 monomial evaluations at coordinate x , $g_X = \bigoplus_{x \in X} g_x$ for its sum over a set X , and $U_J = \text{span}\{g_j : j \in J\}$ for the span over the deleted set J . We assume that the columns indexed by J are independent, as they are in the puncture searches below, so each deleted coordinate contributes one output. A retained fault support X is accepted and harmful (Sec. III A) exactly when $0 \neq g_X \in U_J$. For a given support X , this requires a test of nonzero membership in U_J .

Theorem 3 (downward closure). *If a puncture set J is enlarged to $J \cup L$ with the columns $\{g_l\}_{l \in L}$ independent modulo $U_J = \text{span}\{g_j : j \in J\}$, the span of the deleted columns, so that each new coordinate is a new output, then $d(J \cup L) \leq d(J)$, where $d(J)$ is the distance of the factory punctured at J . Hence distance-preserving punctures are downward closed.*

Proof. Let X be a minimum accepted harmful support for J , so $0 \neq g_X \in U_J$, and put $R = X \cap L$. If $R = \emptyset$, X remains accepted and harmful. If $0 < |R| < |X|$ the residual $X \setminus R$ has column sum $g_X + g_R$ in the enlarged span, nonzero because independence makes every nonempty sum of new columns nonzero modulo U_J , so it is accepted, harmful and shorter. If $R = X$ then $g_X \in U_J$ contradicts that independence. $\square$

Proposition 4 (parity lift). *Let G be a reduced, full-rank $[[n, k, d]]$ circuit with check span S , output rows $g_1, \dots, g_k$ and odd distance d . Suppose its target is homogeneous: every degree-one and degree-two output moment vanishes, so the output is a product of CCZ gates on triples that need not be disjoint. If n is even and $\mathbf{1} \notin \text{span}(S, g_1, \dots, g_k)$, appending the all-ones row $\mathbf{1}$ as a check gives a factory for the same target with n inputs, one more check wire and distance at least $d + 1$. If n is odd, first append a column supported only on the new check wire. The result has $n + 1$ inputs, one more check wire and distance at least $d + 1$.*

Proposition 5 (free-row appending). *Let G be a reduced, full-rank distance- ≥ 3 factory with check span S and output rows $g_1, \dots, g_k$, and prescribe the moments a new output is to have. These are a degree-one value t , degree-two values v_i against each g_i and degree-three values w_{ij} against each pair g_i, g_j with $i < j$, all in $\mathbb{F}_2$. The rows $a \in \mathbb{F}_2^n$ with $\text{wt}(a) \equiv t$, $\text{wt}(a \odot g_i) \equiv v_i$, $\text{wt}(a \odot g_i \odot g_j) \equiv w_{ij}$ and $\text{wt}(a \odot s) \equiv \text{wt}(a \odot s \odot s') \equiv \text{wt}(a \odot s \odot g_i) \equiv 0 \pmod{2}$ for all $s, s' \in S$ form an affine subspace of $\mathbb{F}_2^n$, possibly empty, and those outside the span of the existing check and output rows extend G to a reduced, full-rank distance- ≥ 3 circuit with $k + 1$ output rows and the prescribed moments. For any row h , define its moment vector relative to the old outputs by*

$$M(h) = (\text{wt}(h), (\text{wt}(h \odot g_i))_i, (\text{wt}(h \odot g_i \odot g_j))_{i < j}) \pmod{2}.$$

Suppose no nonzero combination of the old output rows has zero moment vector. If

$$(t, (v_i)_i, (w_{ij})_{i < j}) \neq M\left(\sum_{i \in I} g_i\right) \quad \text{for every } I \subseteq \{1, \dots, k\},$$

including $I = \emptyset$, then no nonzero combination of the extended output rows is idle. The extended circuit is therefore a $[[n, k + 1, \geq 3]]$ factory with no idle output direction.

Proof of Proposition 4. A row c may be appended as a check exactly when $\text{wt}(c)$, $\text{wt}(c \odot g)$ and $\text{wt}(c \odot g \odot g')$ are even for all existing rows g, g' , since a check carries no non-Clifford phase. For $c = \mathbf{1}$ these are its length, $\text{wt}(g)$ and $\text{wt}(g \odot g')$, which for output rows are the degree-one and degree-two moments and vanish by homogeneity, and for checks are even already, since no monomial touching a check wire carries a non-Clifford phase. Thus $\mathbf{1}$ is a legal check when its length is even. In the even- n case the added row is independent of all existing rows by hypothesis, while appending the same bit to every column preserves distinctness and nonzeroness.

In the odd- n case, extend every old row by a zero at the appended column and take $c = \mathbf{1}$ of length $n + 1$. The new column is distinct because the old circuit has no zero column, and c is independent of all old rows because it is the only row nonzero at the new coordinate. The new column enters no overlap of the old rows and has zero output label, so the target is unchanged.

In either case every column has a one on the new check wire, and hence every accepted fault set has even size. Such a set is accepted by the old checks as well. In the even- n case it is therefore an accepted harmful set of G , so it cannot have size below d or size d , since d is odd. In the odd- n case, let e denote the appended column. If an accepted harmful set F excludes e , then it is an accepted harmful set of G of even size and therefore has $|F| \geq d + 1$. If $e \in F$, then $F \setminus \{e\}$ is an accepted harmful set of G , so $|F| - 1 \geq d$. Thus the new distance is at least $d + 1$ in both cases. $\square$

When $d(G) = d$ exactly, $\mathbf{1}$ is automatically independent of the check rows, since otherwise every accepted set would already have even size. Only its independence from the output rows needs to be checked.

Proof of Proposition 5. Each condition is the parity of the coordinatewise product of a with a fixed vector, hence an affine-linear equation over $\mathbb{F}_2$, and a finite system of such equations has an affine solution set. Appending an output row changes no syndrome, so Theorem 1 keeps the distance at least three, and the columns stay distinct and nonzero. A row outside the span of the existing rows increases the row rank by one and hence adds one logical qubit. A row inside that span does not. The moment map M is linear over $\mathbb{F}_2$. Every output direction involving the appended row has the form $a + \sum_{i \in I} g_i$, whose moment vector is

$$(t, (v_i)_i, (w_{ij})_{i < j}) + M\left(\sum_{i \in I} g_i\right).$$

The stated condition makes each such vector nonzero. Output directions not involving a are nonidle by hypothesis. To see that M also detects idleness in the full extended space, write a direction involving the new row as $h = a + s$, with s in the old output span. Since $h \odot h = h$, for every old output-span vector u we have

$$\text{wt}(h \odot a) \equiv \text{wt}(h) + \text{wt}(h \odot s), \quad \text{wt}(h \odot a \odot u) \equiv \text{wt}(h \odot u) + \text{wt}(h \odot s \odot u) \pmod{2}.$$

The terms on the right are linear combinations of components of $M(h)$. Multilinearity then covers every extended output-span vector. Hence no nonzero direction is idle in any output basis. $\square$

Appendix B: Puncture searches

For a parent with hundreds of coordinates, full enumeration of output labelings is beyond our search capacity. We instead fix a base code and search over coordinates to delete, so that each independent puncture turns one input into one output. The first search uses RM(3, 10) on 1024 coordinates, one of the two Reed–Muller parents used in Ref. [20] and the base of its large-block table. We keep a puncture set only if the verifier certifies the required lower bound on the distance.

Our search differs from Ref. [20] in how the puncture set is varied. Haah and Hastings searched random punctures of the smaller RM(2, 7) parent. For RM(3, 10), they began with a randomly punctured distance-three code and successively unpunctured coordinates to raise the distance to four and five. Their distance-six and distance-seven codes were obtained by further unpuncturing their best-rate distance-five code. We run in the opposite direction, deleting further from a puncture set already in hand while holding the distance fixed. Downward closure (Theorem 3) prunes these extensions. A coordinate whose deletion alone breaks the bound cannot appear in any admissible extension, and a pair that breaks it cannot appear together, so single and pairwise tests reduce the search to cliques of a compatibility graph. Starting from the Haah–Hastings puncture set [[887, 137, 5]], the scan identifies 37 safe coordinates among the 887 unpunctured coordinates. They form a 37-vertex, 469-edge compatibility graph. An exact audit of all 5,314 cliques of size at least seven finds 25 admissible sets of size seven and none larger. This certifies $k \leq 144$ within this descendant family, with the bound attained by [[880, 144, 5]] at $\gamma = 1.125$. A separate search found the distance-five entry of Table III, [[879, 145, 5]] at $\gamma = 1.120$. It is another puncture set of the same base code, but not a descendant of the [[887, 137, 5]] seed. Puncturing fewer coordinates can raise the distance instead. With 122 punctures, the base code yields [[902, 122, 6]]. Relative to [[880, 144, 5]], it uses 22 more inputs and produces 22 fewer outputs, with $\gamma = 1.117$. A *ruin-and-recreate move* deletes part of a puncture set and rebuilds it. Restoring four of the 122 coordinates and deleting five others gives [[901, 123, 6]] with $\gamma = 1.111$. Its leading harmful-fault count is larger, with 2781 accepted harmful fault sets of weight six against 2475.

Some of the lowest exponents in the catalog come from puncturing other parents. These include the weighted Reed–Muller codes of Ref. [65] and the derivative four-block Schur-orthogonal extensions. Lemma 2.3 of Ref. [31] gives an

independent derivation of the required Clifford phase corrections. The downset construction of Proposition 3.7 in Ref. [65] restricts the puncture set itself. If it contains a Boolean vector, it also contains every coordinatewise smaller vector, which yields a closed-form distance formula. Our graph search allows arbitrary independent puncture sets, so we certify each distance directly from the resulting columns. Two further moves are used. A *logical-pair check* deletes two output rows and appends their sum as a check. A *paired-column contraction* restricts the row space so that two chosen columns agree, then deletes those columns and a check row that separates them. Successful contractions save two inputs while preserving the verified output count and distance. The distance-six entry of Table III uses both, and the largest factory of this kind in our catalog, $[[1715, 287, 6]]$, reaches $\gamma = 0.998$, the smallest explicit pure-T factory with an exponent below one that we know of.

Appendix C: A worked round

Figure 6 follows the second round of one slot-search campaign that improved the best known input counts in its search cells. The top box is the brief of Sec. VI, fixed for the whole campaign, and everything below it is that round's state. Two words in the figure are the campaign's own. A *geometry* is a slot specification, naming an output count k , an ordered list of check blocks and a permutation group acting on each block. A *cell* is one combination of output count, output gate and distance, for which the campaign keeps a best known input count.

Eighty-four geometries ran in this round. The summary line counts geometries and the direction below it counts cells, which is why the two read differently. Six geometries beat the standing record, in two cells, four of them in the CCZ cell, and four more opened one cell that had been empty. Three of the classes this paper reports come from the round. $[[64, 3, 4]]$ for CCZ improves on the catalog's earlier $[[66, 3, 4]]$, and the round also returned $[[60, 2, 4]]$ for $T^{\otimes 2}$ and $[[48, 2, 4]]$ for CS. All three were found under a cap rather than proved minimal, and the review recorded that at the time.

The rest of the round consisted of unsuccessful solver outcomes. Forty-five geometries returned UNSAT in seconds, ruling out solutions within the specified constraints (Sec. VI). The review compared this distance-four sweep with the previous round's hundred-plus distance-three runs, all of which had returned UNKNOWN at the time limit. On this evidence, the reviewer restricted the next round to distance four and above.

Appendix D: Full catalog

Tables V and VI print the level-three, distance- ≥ 3 factories our searches produced, ordered by input count. There are 699 classes, a class being defined as in Sec. II. A class reached by several searches appears once, represented by the circuit of fewest wires.

Of the 699 classes, 414 have pure- $T^{\otimes k}$ outputs. Table V lists 220 representatives, giving their parameters $[[n, k, d]]$ and width N . Their phase polynomial is $x_0 + \dots + x_{k-1}$ and their T-count is k . Only the highest k at each n and d is printed, since discarding outputs trivially gives smaller k . Table VI lists the other 285 classes, including their output phase polynomials except where summarized below.

The *output phase polynomial* specifies the delivered gate up to Clifford corrections. It contains the same data as the target vector D of Sec. III. Its degree-one, degree-two and degree-three terms are respectively the T, CS and CCZ content of the output gate. We write it in one notation throughout. The wires of a factor are separated by spaces, and + joins the factors. The number of wires in a factor gives its type, so one wire is a T, two a CS and three a CCZ. Thus $0+1+0$ is $T_0T_1CS_{01}$, $0\ 1\ 2$ is CCZ_{012} , and $0\ 64\ 83$ is $CCZ_{0,64,83}$. A polynomial containing only one-wire factors is a product of T gates. Two- and three-wire factors specify CS and CCZ gates. The wires are written out rather than run together because k reaches 84 in Table VI, where 10 would be unreadable as either wire ten or the pair $\{1, 0\}$.

The remaining columns are the width $N = k + r$ and the T-count of the output gate [64]. A dash marks an output wider than $k = 6$, beyond the range of our minimization. Nine output polynomials have dozens to thousands of factors, the largest 3587. Those rows record the number of factors instead, and the gate itself is in the database. A distance written $\geq d$ is a proved lower bound rather than a measured value. The tables include rediscoveries alongside new results. The novelty criterion and class counts are given in Sec. II.

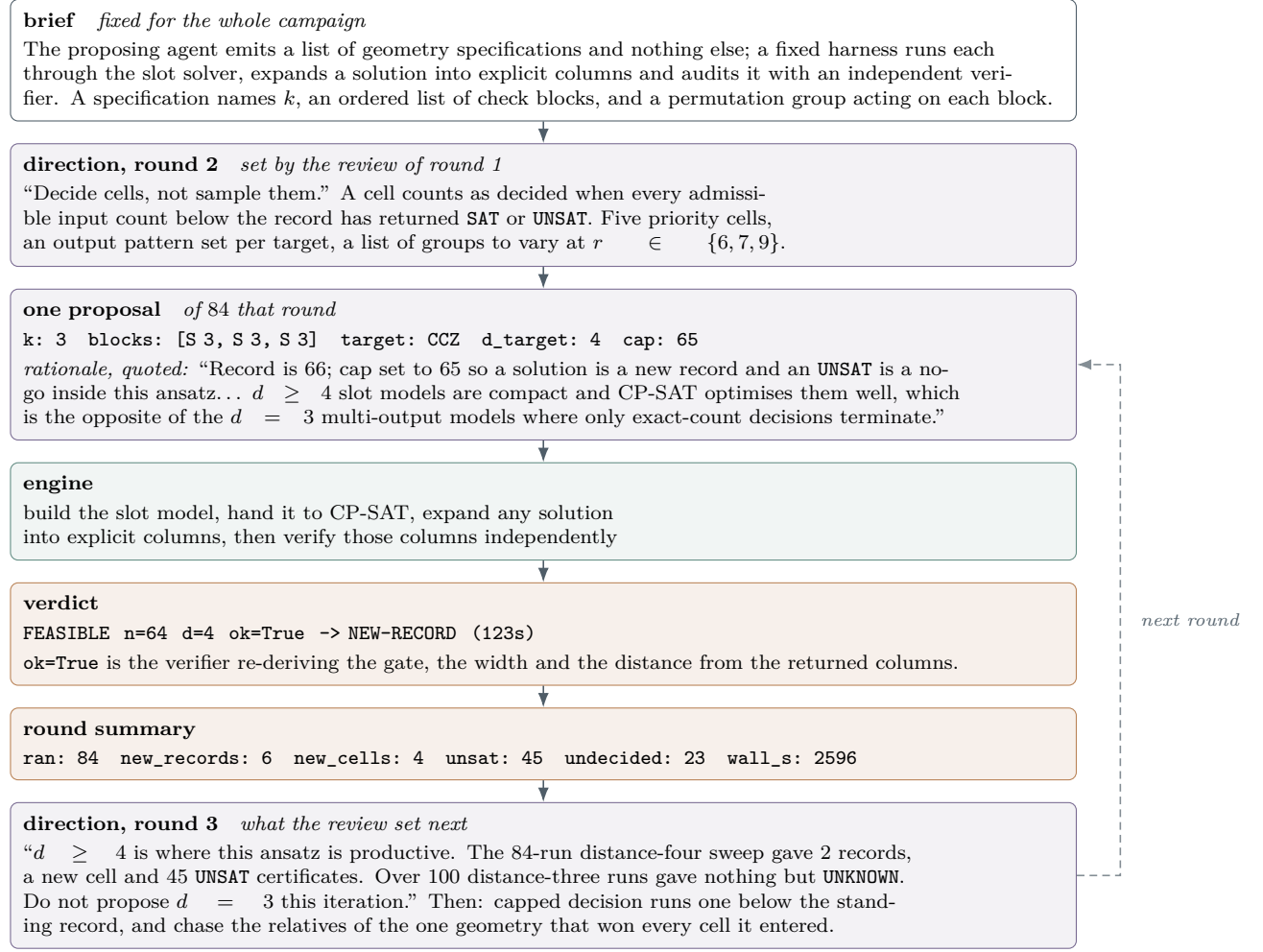


FIG. 6. One logged round of the agent loop, quoted from the campaign’s brief, its direction files, its proposal set and its run log. The brief is fixed for the whole campaign. Everything below it belongs to this round. The proposal shown is one of 84, and it returned a factory at $n = 64$ where the catalog held 66. Most of the round’s runs failed cheaply, forty-five UNSAT in seconds. That is what let the review close the next round to distance four and above. The same round produced the $[[60, 2, 4]]$ and $[[48, 2, 4]]$ records.

TABLE V. Factories with a pure $T^{\otimes k}$ output: k independent magic states. Only the highest k at each n and d is listed.

#	factory	N	#	factory	N	#	factory	N
1	[[15, 1, 3]]	5	47	[[66, 8, 3]]	16	93	[[127, 1, 7]]	22
2	[[23, 1, 3]]	7	48	[[67, 9, 3]]	16	94	[[127, 5, 5]]	19
3	[[27, 1, 3]]	7	49	[[68, 10, 3]]	17	95	[[127, 23, 3]]	30
4	[[28, 2, 3]]	9	50	[[69, 9, 3]]	17	96	[[135, 17, 3]]	25
5	[[29, 1, 3]]	9	51	[[70, 10, 3]]	18	97	[[141, 2, 4]]	12
6	[[30, 2, 3]]	10	52	[[72, 1, 4]]	10	98	[[154, 28, 3]]	37
7	[[31, 1, 3]]	6	53	[[72, 10, 3]]	17	99	[[155, 21, 3]]	31
8	[[32, 2, 3]]	9	54	[[73, 1, 4]]	10	100	[[156, 28, 3]]	38
9	[[33, 1, 3]]	9	55	[[74, 10, 3]]	18	101	[[157, 29, 3]]	38
10	[[34, 2, 3]]	10	56	[[76, 1, 4]]	10	102	[[158, 28, 3]]	39
11	[[35, 3, 3]]	9	57	[[76, 12, 3]]	19	103	[[159, 29, 3]]	39
12	[[36, 2, 3]]	9	58	[[77, 1, 4]]	11	104	[[160, 30, 3]]	40
13	[[37, 1, 3]]	9	59	[[78, 12, 3]]	22	105	[[161, 31, 3]]	41
14	[[38, 2, 3]]	10	60	[[79, 13, 3]]	20	106	[[165, 1, 4]]	10
15	[[39, 3, 3]]	9	61	[[80, 8, 3]]	15	107	[[176, 32, 3]]	42
16	[[40, 2, 3]]	9	62	[[82, 12, 3]]	20	108	[[188, 36, 3]]	46
17	[[41, 1, 3]]	9	63	[[84, 1, 4]]	10	109	[[189, 35, 3]]	46
18	[[42, 2, 3]]	12	64	[[84, 12, 3]]	19	110	[[203, 41, 3]]	51
19	[[43, 3, 3]]	10	65	[[85, 1, 5]]	11	111	[[204, 40, 3]]	50
20	[[44, 4, 3]]	11	66	[[86, 12, 3]]	20	112	[[205, 43, 3]]	53
21	[[45, 3, 3]]	11	67	[[87, 13, 3]]	20	113	[[206, 42, 3]]	52
22	[[46, 2, 3]]	10	68	[[88, 14, 3]]	21	114	[[207, 43, 3]]	54
23	[[47, 5, 3]]	11	69	[[90, 14, 3]]	22	115	[[208, 44, 3]]	54
24	[[48, 1, 4]]	11	70	[[91, 15, 3]]	22	116	[[210, 46, 3]]	56
25	[[48, 4, 3]]	11	71	[[92, 14, 3]]	21	117	[[211, 43, 3]]	55
26	[[49, 1, 5]]	12	72	[[93, 11, 3]]	19	118	[[212, 42, 3]]	55
27	[[49, 3, 3]]	11	73	[[94, 14, 3]]	22	119	[[213, 41, 3]]	55
28	[[50, 4, 3]]	14	74	[[95, 15, 3]]	22	120	[[214, 40, 3]]	55
29	[[51, 5, 3]]	12	75	[[96, 12, 3]]	19	121	[[225, 35, 3]]	43
30	[[52, 1, 4]]	10	76	[[97, 11, 3]]	19	122	[[248, 8, 6]]	53
31	[[52, 5, 3]]	12	77	[[98, 14, 3]]	22	123	[[252, 46, 3]]	59
32	[[53, 5, 3]]	13	78	[[99, 15, 3]]	22	124	[[255, 9, 5]]	25
33	[[54, 4, 3]]	12	79	[[100, 16, 3]]	23	125	[[255, 47, 3]]	62
34	[[55, 7, 3]]	14	80	[[101, 11, 3]]	19	126	[[261, 59, 3]]	69
35	[[56, 8, 3]]	15	81	[[103, 17, 3]]	24	127	[[288, 64, 3]]	75
36	[[57, 5, 3]]	13	82	[[107, 19, 3]]	28	128	[[300, 68, 3]]	79
37	[[58, 6, 3]]	14	83	[[109, 19, 3]]	29	129	[[308, 70, 3]]	81
38	[[59, 7, 3]]	14	84	[[110, 18, 3]]	29	130	[[309, 71, 3]]	82
39	[[60, 2, 4]]	11	85	[[111, 17, 3]]	24	131	[[310, 72, 3]]	83
40	[[60, 8, 3]]	15	86	[[112, 16, 3]]	29	132	[[311, 73, 3]]	84
41	[[61, 7, 3]]	15	87	[[114, 14, 3]]	29	133	[[312, 70, 3]]	83
42	[[62, 8, 3]]	16	88	[[116, 12, 4]]	29	134	[[324, 76, 3]]	87
43	[[63, 11, 3]]	17	89	[[117, 5, 4]]	15	135	[[336, 80, 3]]	91
44	[[64, 2, 4]]	11	90	[[118, 10, 4]]	29	136	[[358, 86, 3]]	97
45	[[64, 8, 3]]	15	91	[[119, 17, 3]]	25	137	[[359, 87, 3]]	98
46	[[65, 9, 3]]	17	92	[[125, 3, 5]]	29	138	[[360, 88, 3]]	99

TABLE V (continued)

#	factory	N	#	factory	N	#	factory	N
139	[[386, 94, 3]]	105	167	[[655, 177, 3]]	189	195	[[887, 137, 5]]	176
140	[[399, 77, 3]]	98	168	[[679, 185, 3]]	197	196	[[896, 128, 6]]	189
141	[[404, 100, 3]]	111	169	[[684, 132, 3]]	168	197	[[901, 123, 6]]	176
142	[[405, 101, 3]]	112	170	[[691, 189, 3]]	201	198	[[902, 122, 6]]	176
143	[[406, 102, 3]]	113	171	[[702, 194, 3]]	205	199	[[904, 120, ≥ 6]]	176
144	[[407, 73, 4]]	92	172	[[715, 197, 3]]	209	200	[[905, 119, ≥ 6]]	176
145	[[407, 101, 3]]	113	173	[[727, 201, 3]]	213	201	[[906, 118, ≥ 6]]	176
146	[[408, 104, 3]]	115	174	[[742, 36, 8]]	78	202	[[907, 117, ≥ 6]]	176
147	[[409, 101, 3]]	114	175	[[750, 210, 3]]	221	203	[[908, 116, ≥ 6]]	176
148	[[410, 100, 3]]	114	176	[[756, 32, 9]]	80	204	[[909, 115, ≥ 6]]	176
149	[[426, 80, 4]]	99	177	[[800, 224, 3]]	237	205	[[909, 171, 3]]	225
150	[[427, 81, 4]]	100	178	[[825, 199, 3]]	237	206	[[910, 114, ≥ 6]]	176
151	[[427, 85, 3]]	115	179	[[846, 122, 6]]	161	207	[[911, 113, ≥ 6]]	176
152	[[428, 82, 4]]	101	180	[[848, 122, 6]]	162	208	[[912, 112, 6]]	176
153	[[429, 83, 4]]	102	181	[[850, 128, 6]]	166	209	[[935, 89, ≥ 6]]	176
154	[[430, 82, 4]]	102	182	[[852, 128, 6]]	167	210	[[936, 88, ≥ 6]]	176
155	[[431, 81, 4]]	102	183	[[854, 128, 6]]	168	211	[[937, 87, ≥ 6]]	176
156	[[432, 80, 4]]	102	184	[[856, 128, 6]]	169	212	[[959, 65, ≥ 6]]	176
157	[[460, 116, 3]]	128	185	[[858, 128, 6]]	170	213	[[960, 184, 3]]	212
158	[[495, 99, 3]]	118	186	[[860, 128, 6]]	171	214	[[988, 36, ≥ 6]]	176
159	[[496, 16, 6]]	78	187	[[862, 162, 3]]	176	215	[[998, 26, ≥ 6]]	176
160	[[502, 92, 3]]	117	188	[[863, 161, 3]]	176	216	[[1023, 31, 5]]	51
161	[[508, 132, 3]]	143	189	[[870, 154, 4]]	176	217	[[1675, 373, 4]]	397
162	[[511, 9, ≥ 6]]	36	190	[[871, 153, 4]]	176	218	[[1676, 372, 4]]	397
163	[[511, 17, 5]]	35	191	[[872, 152, 4]]	176	219	[[1683, 365, 4]]	397
164	[[511, 89, 3]]	98	192	[[879, 145, 5]]	176	220	[[1715, 287, 6]]	329
165	[[558, 146, 3]]	158	193	[[880, 144, 5]]	176			
166	[[606, 162, 3]]	174	194	[[880, 176, 3]]	209			

TABLE VI. Factories with other outputs, deduplicated up to CNOT+ S output equivalence.

#	factory	N	output phase polynomial	T-count
1	[[31, 2, 3]]	7	0+1+0 1	1
2	[[31, 3, 3]]	8	0+1+2+0 1+0 2+1 2+0 1 2	1
3	[[31, 4, 3]]	9	0+1+2+3+0 1+0 2+0 3+1 2+1 3+2 3+0 1 2+0 1 3+0 2 3+1 2 3	1
4	[[31, 5, 3]]	10	0+1+2+3+4+0 1+0 2+0 3+0 4+1 2+1 3+1 4+2 3+2 4+3 4+0 1 2+ 0 1 3+0 1 4+0 2 3+0 2 4+0 3 4+1 2 3+1 2 4+1 3 4+2 3 4	1
5	[[35, 2, 3]]	8	0+1+0 1	1
6	[[35, 2, 3]]	8	0 1	3
7	[[35, 3, 3]]	9	0+1+2+0 1+0 2+1 2+0 1 2	1
8	[[36, 2, 3]]	9	0+1+0 1	1
9	[[36, 2, 3]]	8	0 1	3
10	[[36, 3, 3]]	10	0+1+2+0 1	2
11	[[36, 3, 3]]	10	0+1+2+0 1+0 2+1 2+0 1 2	1
12	[[36, 3, 3]]	9	0 1+0 2	4
13	[[36, 4, 3]]	11	0+1+2+3+0 1+2 3	2
14	[[37, 2, 3]]	10	0+1+0 1	1
15	[[37, 3, 3]]	11	0+1+2+0 1+0 2+1 2+0 1 2	1
16	[[38, 2, 3]]	10	0+1+0 1	1
17	[[38, 3, 3]]	11	0+1+2+0 1	2
18	[[38, 3, 3]]	11	0+1+2+0 1+0 2+1 2+0 1 2	1
19	[[38, 4, 3]]	12	0+1+2+3+0 1+2 3	2
20	[[43, 2, 3]]	9	0 1	3
21	[[43, 3, 3]]	10	0+0 1+0 2	5
22	[[43, 3, 3]]	10	0+1 2	4
23	[[43, 4, 3]]	11	0+1 2+1 3	5
24	[[44, 3, 3]]	10	0+1 2	4
25	[[45, 2, 3]]	10	0 1	3
26	[[47, 3, 3]]	9	0 1 2	7
27	[[47, 4, 3]]	10	0+1+2 3	5
28	[[47, 4, 3]]	10	0+1 2 3	7
29	[[47, 4, 3]]	10	0 1+2 3	6
30	[[47, 5, 3]]	11	0+1+4+0 1+0 4+1 4+0 1 2+0 1 3+0 2 3+0 2 4+1 2 4+2 3 4	10
31	[[47, 6, 3]]	12	0+1+2+3+0 1+0 2+0 3+1 2+1 3+2 3+0 1 2+0 1 3+0 1 4+0 1 5+ 0 2 3+0 2 4+0 2 5+0 3 4+0 3 5+0 4 5+1 2 3+1 2 4+1 2 5+1 4 5	11
32	[[48, 2, 4]]	10	0 1	3
33	[[48, 3, 4]]	11	0 1+0 2+0 1 2	3
34	[[48, 3, 3]]	9	0 1 2	7
35	[[48, 3, 4]]	10	0 1 2	7
36	[[48, 4, 4]]	12	0 1+0 2+0 3+0 1 2+0 1 3+0 2 3	3
37	[[48, 4, 3]]	10	0 1+2 3	6
38	[[51, 4, 3]]	11	0+1+2 3	5
39	[[52, 2, 4]]	11	0+1+0 1	1
40	[[52, 2, 3]]	9	0 1	3
41	[[55, 3, 3]]	10	0 1 2	7
42	[[55, 4, 3]]	11	0+1+2 3	5

TABLE VI (continued)

#	factory	N	output phase polynomial	T-count
43	[[55, 4, 3]]	11	0+1 2 3	7
44	[[55, 6, 3]]	13	0+1+2+3+4+0 1+0 2+1 2+1 4+2 3+3 4+0 1 2+0 1 3+0 2 4+0 3 5+ 0 4 5+1 3 5+2 4 5+3 4 5	9
45	[[55, 6, 3]]	13	0+1+2+3+4+5+0 1+0 2+0 3+0 4+0 5+1 2+1 3+1 4+1 5+2 3+2 4+2 5+ 0 1 2+3 4 5	9
46	[[55, 6, 3]]	13	0+1+2+3+4+5+0 1+0 2+0 3+0 4+0 5+1 2+1 3+1 4+1 5+2 3+2 4+ 2 5+3 4+3 5+0 1 2+0 1 3+0 1 4+0 1 5+0 2 3+0 2 4+0 2 5+0 3 4+ 0 3 5+1 2 3+1 2 4+1 3 5+2 4 5+3 4 5	11
47	[[55, 6, 3]]	13	0+1+2+3+4+5+0 1+0 2+0 3+0 4+1 2+1 3+1 4+2 3+2 4+2 5+3 4+ 0 1 2+0 1 3+0 1 4+0 1 5+0 2 3+0 2 4+0 3 5+0 4 5+1 2 3+1 2 4+ 2 3 4+3 4 5	9
48	[[55, 6, 3]]	13	0+1+2+3+5+0 1+0 2+0 3+1 5+2 5+3 5+0 4 5+1 2 4+1 3 4+2 3 4	7
49	[[55, 6, 3]]	13	0+1+2+4+5+0 1+0 4+0 1 2+0 1 3+0 2 3+0 2 4+0 2 5+0 3 4+0 3 5+ 1 2 4+1 2 5+1 3 5+1 4 5+2 3 5+2 4 5+3 4 5	9
50	[[55, 6, 3]]	13	0+2+4+0 1+0 2+0 3+1 4+0 1 2+0 1 3+0 1 4+0 1 5+0 2 3+0 2 4+ 0 3 5+1 2 4+1 2 5+1 3 4+1 3 5+1 4 5+2 3 4+2 3 5+3 4 5	9
51	[[56, 2, 3]]	9	0 1	3
52	[[56, 5, 4]]	13	0 1 2+0 3 4	11
53	[[56, 6, 3]]	13	0+2+3+4+5+0 1+0 2+0 3+0 4+1 2+1 3+1 4+1 5+2 3+2 4+3 4+0 1 2+ 0 1 3+0 1 4+0 2 5+0 3 5+0 4 5+1 2 3+1 2 4+1 3 4+2 3 5+2 4 5+ 3 4 5	10
54	[[56, 6, 3]]	13	0+3+4+5+0 1+0 2+2 3+2 4+2 5+3 4+3 5+4 5+0 1 2+0 1 3+0 1 4+ 0 1 5+0 3 4+1 3 4+2 3 4+2 3 5+2 4 5	8
55	[[59, 4, 3]]	11	0+1+2 3	5
56	[[59, 5, 3]]	12	0+0 1+0 1 2+1 3 4	9
57	[[59, 5, 3]]	12	0+0 1+2 3+0 1 2+0 2 3+1 3 4	9
58	[[59, 5, 3]]	12	0+0 1+2 4+0 2 3	9
59	[[59, 5, 3]]	12	0+0 1+3 4+0 1 2+2 3 4	7
60	[[59, 5, 3]]	12	0+1 2+3 4	7
61	[[59, 5, 3]]	12	0+1 2+3 4+0 1 2+0 1 3+0 2 4+1 2 3	10
62	[[59, 5, 3]]	12	0+1 3+0 1 2+1 3 4	9
63	[[59, 5, 3]]	12	0+1 3+2 4+0 1 2+0 1 3+0 2 4+1 2 3	9
64	[[59, 5, 3]]	12	0 1+2 3 4	9
65	[[59, 6, 3]]	13	0+0 1+1 2+1 3+1 4+1 5+2 3+2 4+2 5+3 4+3 5+4 5+0 2 3+0 2 4+ 0 2 5+0 3 4+0 3 5+0 4 5	7
66	[[59, 6, 3]]	13	0+1+2+3+4+0 1+0 2+0 4+1 3+2 3+3 4+0 1 2+0 1 3+0 1 4+0 2 3+ 0 2 5+0 3 4+0 3 5+0 4 5+1 2 3+1 3 5+2 3 4+3 4 5	11
67	[[59, 6, 3]]	13	0+1+2+3 4 5	9
68	[[59, 6, 3]]	13	0+1+2+4+5+0 1+0 2+2 4+2 5+0 1 2+0 1 3+0 1 4+0 1 5+0 2 3+ 0 3 4+0 3 5+1 2 4+1 3 5+1 4 5+2 3 5+3 4 5	11
69	[[60, 4, 3]]	11	0+1+2 3	5
70	[[60, 5, 3]]	12	0+0 1+0 1 2+0 3 4	9
71	[[60, 5, 3]]	12	0+0 1+0 1 2+2 3 4	9
72	[[60, 5, 3]]	12	0+0 1+0 2 3+1 2 4	10
73	[[60, 5, 3]]	12	0+0 1+2 3+0 1 2+1 3 4	10
74	[[60, 5, 3]]	12	0+0 1+2 4+0 2 3+2 3 4	9
75	[[60, 5, 3]]	12	0+0 1+2 4+1 2 3	8
76	[[60, 5, 3]]	12	0+0 1+3 4+0 1 2	8
77	[[60, 5, 3]]	12	0+0 1+3 4+0 1 2+0 1 3+1 2 4	8
78	[[60, 5, 3]]	12	0+0 1+3 4+0 1 2+0 2 3+1 2 4	10
79	[[60, 5, 3]]	12	0+0 1+3 4+0 1 2+0 2 3+2 3 4	8
80	[[60, 5, 3]]	12	0+0 1+3 4+0 1 2+1 2 3	9
81	[[60, 5, 3]]	12	0+0 1+3 4+0 1 2+1 3 4	9
82	[[60, 5, 3]]	12	0+0 1+3 4+0 2 3+1 2 4	9
83	[[60, 5, 3]]	12	0+1 2+1 3 4	8
84	[[60, 5, 3]]	12	0+1 2+3 4+0 1 2+0 1 3	9

TABLE VI (continued)

#	factory	N	output phase polynomial	T-count
85	[[60, 5, 3]]	12	0+1 2+3 4+0 1 2+0 1 3+0 2 4	10
86	[[60, 5, 3]]	12	0+1 2+3 4+0 1 2+0 1 3+0 2 4+0 3 4	9
87	[[60, 5, 3]]	12	0+1 3+0 1 2+0 3 4+1 2 3	10
88	[[60, 5, 3]]	12	0+1 3+0 1 2+2 3 4	10
89	[[60, 5, 3]]	12	0+1 3+2 4+0 1 2+0 3 4	10
90	[[60, 5, 3]]	12	0+1 3+2 4+0 1 2+1 2 3	9
91	[[60, 5, 3]]	12	0+3 4+0 1 2	9
92	[[60, 5, 3]]	12	0+3 4+0 1 2+0 1 3+1 2 4	9
93	[[60, 5, 3]]	12	0+3 4+0 1 2+1 2 3	8
94	[[60, 5, 3]]	12	0+3 4+0 1 2+1 3 4	8
95	[[60, 5, 3]]	12	0 1+0 1 2+2 3 4	9
96	[[60, 5, 3]]	12	0 1+3 4+0 1 2+0 3 4	8
97	[[60, 6, 3]]	13	0+0 1+0 2+0 3+0 4+0 5+0 1 2+0 1 3+0 2 4+1 2 3+1 2 5+1 3 4+ 1 3 5+1 4 5	10
98	[[60, 6, 3]]	13	0+1+2+0 1+0 2+0 3+1 4+2 4+3 4+3 5+4 5+0 1 2+0 1 3+0 1 4+ 0 2 3+0 2 5+0 4 5+1 3 5+2 3 4+2 4 5	8
99	[[60, 6, 3]]	13	0+1+2+3+4+0 1+0 2+0 5+1 2+1 5+2 5+3 5+4 5+0 1 2+0 1 3+0 1 4+ 0 2 3+0 2 4+0 3 4+1 2 3+1 2 4+1 3 5+2 3 5	9
100	[[60, 6, 3]]	13	0+1+2+4+5+0 1+0 2+0 3+1 3+1 5+2 3+3 4+3 5+0 1 2+0 1 3+0 1 4+ 0 2 3+0 2 5+0 4 5+1 3 5+1 4 5+2 4 5	10
101	[[60, 6, 3]]	13	0+1+3+4+5+0 1+0 2+0 5+1 2+1 5+2 3+2 4+2 5+0 1 2+0 1 3+0 1 4+ 0 2 5+0 3 5+1 2 5+1 3 5+3 4 5	10
102	[[60, 6, 3]]	13	0+1+4+0 1+0 4+1 5+2 5+4 5+0 1 2+0 1 3+0 2 3+0 2 4+0 2 5+ 0 3 4+0 3 5+1 2 3+1 2 4+1 2 5+2 3 4+2 3 5+2 4 5	8
103	[[60, 6, 3]]	13	0+1+4+2 3+0 1 2+0 1 3+0 1 4+0 2 5+0 3 5+1 2 5+1 3 5+2 3 4+ 2 3 5+2 4 5+3 4 5	8
104	[[60, 6, 3]]	13	0+2+3+4+5+0 1+0 2+0 3+0 4+0 5+1 2+1 3+1 4+1 5+2 5+3 5+4 5+ 0 1 2+0 1 3+0 1 4+0 1 5+0 2 3+0 2 4+1 2 5+1 3 5+1 4 5+2 3 4+ 2 3 5+2 4 5	8
105	[[60, 6, 3]]	13	0+2+3+4+5+0 1+0 2+0 3+0 4+1 2+1 3+1 4+1 5+2 4+2 5+3 4+0 1 2+ 0 1 3+0 1 4+0 2 3+0 2 5+1 2 4+1 2 5+1 3 4+3 4 5	10
106	[[60, 6, 3]]	13	0+2+3+4+5+0 1+0 2+0 3+0 4+1 2+1 3+1 4+1 5+2 5+3 5+4 5+0 1 2+ 0 1 3+0 1 4+0 2 3+0 2 5+1 2 5+1 3 5+1 4 5+2 3 4+2 4 5	10
107	[[60, 6, 3]]	13	0+3+4+0 1+0 2+0 3+0 4+1 3+1 4+2 3+2 4+3 4+3 5+4 5+0 1 2+ 0 1 3+0 1 4+0 1 5+0 2 3+0 2 4+0 3 5+0 4 5+1 2 3+1 2 4+1 3 4+ 2 3 4+2 3 5+2 4 5	10
108	[[60, 6, 3]]	13	0+3+5+0 1+0 2+0 3+0 4+0 5+1 3+1 5+2 3+2 5+3 4+4 5+0 1 2+ 0 1 3+0 1 4+0 1 5+0 2 3+0 2 4+1 2 3+1 2 5+1 3 4+1 4 5+2 3 4+ 2 3 5+2 4 5	12
109	[[60, 6, 3]]	13	0+4+5+0 1+0 2+0 3+1 2+1 3+1 4+1 5+2 3+2 4+2 5+3 4+3 5+0 4 5	8
110	[[60, 6, 3]]	13	0 1+0 2+0 3+0 4+0 5+0 1 2+0 1 3+0 1 4+1 2 3+1 2 4+1 3 4+ 2 3 5+2 4 5+3 4 5	10
111	[[60, 6, 3]]	13	0 1+0 2+0 3+0 5+1 2+2 3+2 4+2 5+0 1 2+0 1 3+0 1 4+0 2 3+ 1 2 5+2 4 5	12
112	[[60, 6, 3]]	13	0 1+0 5+2 3+2 4+0 1 2+0 1 3+0 1 4+0 2 3+0 2 4+0 2 5+0 3 5+ 0 4 5+1 2 3+1 2 4+2 3 5+2 4 5	8
113	[[60, 7, 3]]	14	0+0 1+0 2+0 3+0 4+1 2+1 3+2 3+4 5+5 6+0 1 2+0 1 3+0 1 4+ 0 2 3+0 2 4+0 3 4+1 2 3+1 4 5+1 5 6+2 4 5+2 5 6+3 4 5+3 5 6	—
114	[[60, 7, 3]]	14	<i>entangled output, 37 factors</i>	—
115	[[62, 6, 3]]	16	0+1+2+3+4+0 1+0 2+0 3+0 4+0 5+1 2+1 3+1 4+1 5+2 3+2 4+2 5+ 3 4+3 5+4 5+0 1 2+0 1 3+0 1 4+0 2 3+0 2 4+0 3 4	11
116	[[62, 7, 3]]	17	0+0 1+0 2+0 3+0 4+0 5+0 6+0 1 2+0 1 3+0 1 4+0 1 5+0 1 6+ 0 2 3+0 2 4+0 3 5+0 4 6+0 5 6+2 3 4+2 3 6+2 4 5	—
117	[[63, 2, 3]]	8	0 1	3
118	[[63, 3, 3]]	9	0+1 2	4
119	[[63, 3, 3]]	9	0 1 2	7
120	[[63, 4, 3]]	10	0+1 2 3	7
121	[[63, 4, 3]]	10	0 1+2 3	6
122	[[63, 5, 3]]	11	0+1+2+3 4	6
123	[[63, 5, 3]]	11	0+1 2+3 4	7
124	[[63, 5, 3]]	11	0 1+2 3 4	9
125	[[63, 5, 3]]	11	0 1 2+0 3 4	11
126	[[63, 6, 3]]	16	0+1+2+3+4+0 1+0 2+1 2+1 4+2 3+0 1 2+0 1 3+0 2 4+0 3 4+0 3 5+ 0 4 5+1 3 4+1 3 5+2 3 4+2 4 5	10

TABLE VI (continued)

#	factory	N	output phase polynomial	T-count
127	[[63, 6, 3]]	16	0+1+2+3+4+5+0 1+0 2+0 4+1 2+1 3+1 4+1 5+2 3+2 4+2 5+3 4+4 5+	12
			0 1 2+0 1 3+0 2 3+0 3 4+0 3 5+1 2 4+1 3 4+2 4 5	
128	[[63, 6, 3]]	16	0+1+2+3+5+0 1+0 2+2 5+0 1 2+0 1 3+0 1 4+0 2 3+0 2 4+0 3 4+	11
			0 3 5+1 2 3+1 2 4+1 2 5+1 3 4+1 4 5+2 4 5+3 4 5	
129	[[63, 6, 3]]	16	0+2+3+4+0 1+1 4+2 3+2 4+0 1 2+0 1 3+0 4 5+1 4 5+2 4 5+3 4 5	10
130	[[63, 6, 3]]	12	0 1+2 3+4 5	9
131	[[63, 6, 3]]	12	0 1 2+3 4 5	13
132	[[63, 7, 3]]	17	0+1 2+1 3+1 4+1 5+1 6+1 2 3+1 2 4+1 2 5+1 3 6+1 4 6+2 3 4+	—
			3 4 5+3 4 6	
133	[[63, 7, 3]]	17	0+1 2+1 3+1 4+1 5+2 6+3 6+4 6+5 6+1 2 3+1 2 4+1 2 5+1 2 6+	—
			1 3 4+1 3 5+1 4 6+2 3 5+2 5 6+3 4 5+3 4 6+4 5 6	
134	[[63, 7, 3]]	17	0+1 2+1 3+2 6+3 6+4 5+1 2 3+1 2 4+1 2 5+1 3 4+1 4 5+2 3 6+	—
			2 4 5+2 4 6+2 5 6+3 4 5+3 4 6	
135	[[63, 7, 3]]	17	0+1 2+3 4+1 2 3+1 3 4+1 3 5+1 4 5+1 4 6+2 3 6+2 4 5	—
136	[[63, 7, 3]]	17	0+1 2+3 5+5 6+1 2 3+1 3 4+1 3 5+1 4 5+1 4 6+2 3 4+2 3 6+	—
			2 4 6+2 5 6	
137	[[63, 7, 3]]	13	0+1 2 3+4 5 6	—
138	[[63, 8, 3]]	18	0+1 2+1 2 3+1 4 5+1 4 6+1 5 7+2 4 5+2 6 7	—
139	[[63, 8, 3]]	18	0+1 2+4 6+1 2 3+1 4 5+2 6 7+3 4 6	—
140	[[63, 12, 3]]	18	0+1+2+3+4+5+6+7+8+9 10 11	—
141	[[63, 16, 3]]	22	0+1 2 3+1 4 5+1 6 7+2 8 9+2 10 11+3 12 13+3 14 15+4 8 12+	—
			4 10 14+5 9 13+5 11 15+6 8 15+6 10 13+7 9 14+7 11 12	
142	[[64, 2, 4]]	11	0 1	3
143	[[64, 3, 4]]	10	0 1 2	7
144	[[64, 5, 4]]	12	0 1 2+0 3 4	11
145	[[64, 6, 4]]	13	0 1 2+0 1 3+0 1 4+0 2 3+0 2 4+0 3 5+1 2 3+1 2 5+1 3 5+3 4 5	15
146	[[64, 6, 4]]	13	0 1 2+3 4 5	13
147	[[64, 15, 4]]	22	0 1 2+0 3 4+0 5 6+1 7 8+1 9 10+2 11 12+2 13 14+3 7 11+	—
			3 9 13+4 8 12+4 10 14+5 7 14+5 9 12+6 8 13+6 10 11	
148	[[66, 3, 4]]	12	0 1 2	7
149	[[67, 4, 3]]	11	0+1+2 3	5
150	[[67, 5, 3]]	12	0+1 2+3 4	7
151	[[67, 5, 3]]	12	0 1+2 3 4	9
152	[[67, 6, 3]]	13	0+1+2+3 4 5	9
153	[[68, 4, 3]]	11	0+1+2 3	5
154	[[68, 5, 3]]	12	0+1 2+3 4	7
155	[[68, 5, 3]]	12	0 1+2 3 4	9
156	[[71, 4, 3]]	11	0+1+2 3	5
157	[[71, 5, 3]]	12	0+1 2+3 4	7
158	[[71, 6, 3]]	13	0+1+2+3 4 5	9
159	[[71, 6, 3]]	13	0+1 2+3 4 5	10
160	[[72, 2, 4]]	11	0 1	3
161	[[72, 3, 4]]	11	0 1+0 2	4
162	[[72, 3, 4]]	12	0 1 2	7
163	[[72, 4, 4]]	13	0 1+2 3	6
164	[[72, 5, 3]]	12	0 1+2 3 4	9
165	[[75, 4, 3]]	11	0+1+2 3	5
166	[[75, 5, 3]]	12	0+1 2+3 4	7
167	[[75, 6, 3]]	13	0+1+2+3 4 5	9
168	[[75, 6, 3]]	13	0+1 2+3 4 5	10

TABLE VI (continued)

#	factory	N	output phase polynomial	T-count
169	[[76, 2, 4]]	11	0+0 1	2
170	[[76, 4, 4]]	13	0+1+2+3+0 1+2 3	2
171	[[76, 4, 4]]	13	0 1 2+0 1 3	7
172	[[76, 5, 3]]	12	0+1+2+3 4	6
173	[[76, 5, 3]]	12	0+1+2 3 4	8
174	[[76, 5, 3]]	12	0+1 2+3 4	7
175	[[76, 5, 3]]	12	0 1+2 3 4	9
176	[[77, 3, 4]]	13	0 1 2	7
177	[[77, 4, 4]]	14	0+1 2 3	7
178	[[77, 5, 4]]	15	0 1 2+0 3 4	11
179	[[77, 6, 4]]	16	0 1 2+3 4 5	13
180	[[77, 7, 4]]	17	0+1 2 3+4 5 6	—
181	[[79, 5, 3]]	12	0+1+2+3 4	6
182	[[79, 5, 3]]	12	0+1 2+3 4	7
183	[[79, 6, 3]]	13	0+1+2+3 4 5	9
184	[[79, 6, 3]]	13	0+1 2+3 4 5	10
185	[[79, 7, 3]]	14	0+1 2 3+4 5 6	—
186	[[80, 2, 4]]	10	0 1	3
187	[[80, 3, 4]]	11	0 1+0 2	4
188	[[80, 4, 4]]	12	0 1+2 3	6
189	[[80, 5, 4]]	13	0 1+2 3 4	9
190	[[80, 6, 3]]	13	0 1 2+3 4 5	13
191	[[82, 11, 3]]	19	0+1+2+3+4+5+6+7+8 9 10	—
192	[[83, 4, 3]]	11	0+1+2 3	5
193	[[83, 5, 3]]	12	0+1 2+3 4	7
194	[[83, 6, 3]]	13	0+1+2+3 4 5	9
195	[[83, 6, 3]]	13	0+1 2+3 4 5	10
196	[[84, 3, 4]]	12	0 1 2	7
197	[[84, 5, 3]]	12	0+1+2+3 4	6
198	[[84, 6, 3]]	13	0+1+2 3+4 5	8
199	[[84, 6, 3]]	13	0+1 2+3 4 5	10
200	[[84, 6, 3]]	13	0 1 2+3 4 5	13
201	[[84, 7, 3]]	14	0+1+2+3+4 5 6	—
202	[[85, 2, 5]]	18	0+1+0 1	1
203	[[87, 6, 3]]	13	0+1+2+3 4 5	9
204	[[87, 6, 3]]	13	0+1+2 3+4 5	8
205	[[87, 6, 3]]	13	0+1 2+3 4 5	10
206	[[87, 6, 3]]	13	0 1 2+3 4 5	13
207	[[88, 6, 3]]	13	0+1+2 3+4 5	8
208	[[88, 6, 3]]	13	0+1 2+3 4 5	10
209	[[88, 6, 3]]	13	0 1 2+3 4 5	13
210	[[88, 13, 3]]	20	0+1+2+3+4+5+6+7+8+9+10 11 12	—

TABLE VI (continued)

#	factory	N	output phase polynomial	T-count
211	[[90, 11, 3]]	19	0+1+2+3+4+5+6+7+8 9 10	—
212	[[91, 4, 3]]	11	0+1+2 3	5
213	[[91, 6, 3]]	13	0+1+2+3 4 5	9
214	[[91, 6, 3]]	13	0+1 2+3 4 5	10
215	[[91, 7, 3]]	14	0+1 2+3 4+5 6	—
216	[[91, 7, 3]]	14	0+1 2 3+4 5 6	—
217	[[92, 6, 3]]	13	0+1+2 3+4 5	8
218	[[92, 6, 3]]	13	0+1 2+3 4 5	10
219	[[92, 6, 4]]	14	0 1 2+3 4 5	13
220	[[92, 7, 3]]	14	0+1 2 3+4 5 6	—
221	[[92, 15, 3]]	22	0+1+2+3+4+5+6+7+8+9+10+11+12 13 14	—
222	[[95, 7, 3]]	14	0+1+2 3+4 5 6	—
223	[[95, 7, 3]]	14	0+1 2+3 4+5 6	—
224	[[95, 7, 3]]	14	0+1 2 3+4 5 6	—
225	[[95, 12, 3]]	19	0+1+2+3+4+5+6+7+8+9 10 11	—
226	[[96, 6, 3]]	13	0 1 2+3 4 5	13
227	[[96, 13, 3]]	20	0+1+2+3+4+5+6+7+8 9+10 11 12	—
228	[[99, 9, 3]]	16	0 1 2+3 4 5+6 7 8	—
229	[[99, 12, 3]]	19	0 1 2+3 4 5+6 7 8+9 10 11	—
230	[[99, 13, 3]]	20	0+1 2 3+4 5 6+7 8 9+10 11 12	—
231	[[99, 16, 3]]	23	0+1+2+3+4+5+6+7+8+9+10+11+12+13 14 15	—
232	[[100, 2, 4]]	12	0 1	3
233	[[100, 6, 4]]	15	0 1 2+0 1 3+0 1 4+0 2 3+1 2 5+1 4 5+2 4 5+3 4 5	17
234	[[100, 9, 4]]	17	0 1 2+3 4 5+6 7 8	—
235	[[100, 12, 4]]	20	0 1 2+3 4 5+6 7 8+9 10 11	—
236	[[100, 15, 3]]	23	0+1+2+3+4+5+6+7+8+9+10+11+12 13 14	—
237	[[103, 7, 3]]	14	0+1+2 3+4 5 6	—
238	[[103, 8, 3]]	15	0+1+2+3+4+5 6 7	—
239	[[103, 8, 3]]	15	0+1 2+3 4+5 6 7	—
240	[[104, 7, 3]]	14	0+1+2 3+4 5 6	—
241	[[104, 8, 3]]	15	0+1 2+3 4+5 6 7	—
242	[[111, 4, 3]]	11	0+1+2 3	5
243	[[111, 8, 3]]	15	0+1 2+3 4+5 6 7	—
244	[[111, 9, 3]]	16	0+1+2+3 4 5+6 7 8	—
245	[[112, 12, 3]]	19	0 1 2+3 4 5+6 7 8+9 10 11	—
246	[[112, 16, 3]]	23	0+1+2+3+4 5 6+7 8 9+10 11 12+13 14 15	—
247	[[116, 2, 4]]	12	0 1	3
248	[[116, 3, 4]]	13	0+1 2	4
249	[[117, 2, 4]]	12	0+0 1	2
250	[[119, 20, 3]]	28	0+1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17 18 19	—
251	[[120, 3, 4]]	13	0 1+0 2	4
252	[[120, 4, 4]]	14	0 1 2+0 1 3	7

TABLE VI (continued)

#	factory	N	output phase polynomial	T-count
253	[[127, 12, 3]]	19	0 1 2+3 4 5+6 7 8+9 10 11	—
254	[[127, 16, 3]]	23	0+1 2 3+4 5 6+7 8 9+10 11 12+13 14 15	—
255	[[127, 17, 3]]	24	0+1+2+3+4+5 6 7+8 9 10+11 12 13+14 15 16	—
256	[[127, 26, 3]]	33	0+1+2+3+4+5+6+7+8+9+10+11 12 13+11 14 15+11 16 17+12 18 19+ 12 20 21+13 22 23+13 24 25+14 18 22+14 20 24+15 19 23+ 15 21 25+16 18 25+16 20 23+17 19 24+17 21 22	—
257	[[127, 37, 3]]	44	<i>entangled output</i> , 1868 factors	—
258	[[128, 3, 6]]	18	0 1 2	7
259	[[128, 12, 4]]	20	0 1 2+3 4 5+6 7 8+9 10 11	—
260	[[128, 15, 4]]	23	0 1 2+3 4 5+6 7 8+9 10 11+12 13 14	—
261	[[136, 7, 4]]	16	0 1 2+0 1 3+0 1 4+0 1 5+0 1 6+0 2 3+0 2 4+0 2 5+0 3 4+0 5 6+ 1 2 6+1 3 5+1 3 6+1 4 6+2 3 5+2 3 6+2 4 6+3 4 5+3 4 6+4 5 6	—
262	[[136, 7, 4]]	17	0 1 2+0 1 3+0 1 4+0 1 5+0 2 3+0 2 4+0 3 4+1 2 5+2 3 6+2 5 6+ 3 4 5+3 4 6+3 5 6+4 5 6	—
263	[[160, 4, 4]]	13	0 1+0 2+0 3	7
264	[[172, 3, 4]]	13	0 1+0 2+1 2	3
265	[[176, 2, 5]]	26	0 1	3
266	[[176, 3, 5]]	27	0 1 2	7
267	[[176, 3, 6]]	28	0 1 2	7
268	[[176, 5, 5]]	29	0 1+0 2+0 3+0 4+1 2+1 3+2 4+3 4+0 1 2+0 1 3+0 2 3	8
269	[[176, 9, 4]]	20	<i>entangled output</i> , 41 factors	—
270	[[188, 3, 4]]	13	0 1+0 2	4
271	[[208, 5, 4]]	14	0 1 2+0 3 4	11
272	[[228, 2, 4]]	12	0 1	3
273	[[255, 3, 7]]	38	0 1 2	7
274	[[255, 4, 7]]	39	0+1 2 3	7
275	[[255, 5, 7]]	41	0 1 2+0 3 4	11
276	[[255, 6, 7]]	41	0+1+0 1+0 1 2+0 1 3+0 1 4+0 1 5+0 2 3+0 2 4+0 2 5+1 2 3+ 1 2 4+1 3 5+1 4 5+2 3 5+2 4 5	7
277	[[256, 6, 6]]	23	0 1 2+3 4 5	13
278	[[256, 9, 4]]	24	<i>entangled output</i> , 36 factors	—
279	[[256, 11, 6]]	35	0 1 2+0 1 3+0 1 4+0 1 5+0 1 6+0 1 7+0 1 8+0 1 9+0 1 10	—
280	[[256, 16, 4]]	28	<i>entangled output</i> , 278 factors	—
281	[[256, 84, 4]]	93	<i>entangled output</i> , 1120 factors	—
282	[[496, 36, 4]]	47	<i>entangled output</i> , 3587 factors	—
283	[[511, 15, 5]]	33	<i>entangled output</i> , 68 factors	—
284	[[512, 39, ≥ 6]]	85	0 1 2+3 4 5+6 7 8+9 10 11+12 13 14+15 16 17+18 19 20+ 21 22 23+24 25 26+27 28 29+30 31 32+33 34 35+36 37 38	—
285	[[512, 84, ≥ 6]]	130	<i>entangled output</i> , 280 factors	—